FALL 11

COMPUTER NUMERICAL CONTROL MACHINE 3-AXIS PLOTTER

Anh Luong luong@eng.utah.edu

Willis Lutz lbzinuse@gmail.com

Jared Pringle jaredpringle86@gmail.com

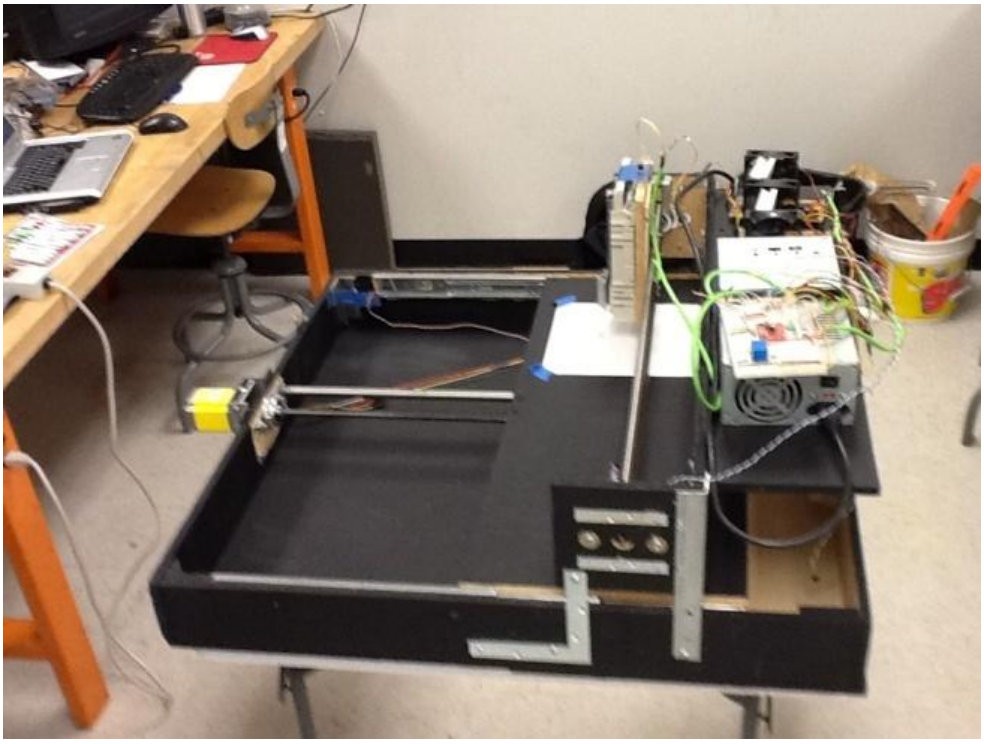
Ashton Snelgrove snelgrov@eng.utah.edu

Website:

http://www.eng.utah.edu/~luong/Anh_Luong/Team_Teal/Team_Teal.html

Table of Contents

TABLE OF CONTENTS **2**
1. INTRODUCTION **4**
2. PROJECT DESCRIPTION **4**
2.1 SOFTWARE 4
2.1.1 Software Overview *4*
2.1.2 Design Considerations *4*
2.1.3 SVG *5*
2.1.4 G-code *5*
2.1.5 Gcodetools *5*
2.1.6 Enhanced Machine Controller *6*
2.2 HARDWARE 6
2.2.1 Hardware Overview *6*
2.2.2 Microcontroller prototyping *6*
2.2.3 Analog Circuit *7*
2.2.4 Stepper Motors *7*
2.2.5 Emergency Stop Relay *8*
2.2.6 Positional Feedback *8*
2.3 MACHINE CONSTRUCTION 8
2.3.1 Frame *8*
2.3.2 X and Y Axes *10*
2.3.3 Z Axis *12*
3. CONCLUSION **13**
4. BILL OF MATERIALS **14**
5. REFERENCES **15**
APPENDIX A: ANALOG CIRCUIT DESIGN **16**
APPENDIX B: FRAME BLUEPRINTS **19**
APPENDIX C: CONFIGURATION AND SOFTWARE **20**
C.1 JOYPAD.HAL 20
C.2 TEAL-MILL.INI 24
C.3 TEAL-MILL.HAL 26
C.4 ARDUINO SERVO PROTOTYPE SKETCH 28
C.5 PATCH FOR GCODETOOLS, BRANCHED OFF REVISION 209. 30



1. Introduction

Computer Numeric Control (CNC) refers to a wide variety of machines which are controlled electronically and have many uses, including milling, drawing, extruding, cutting, and lathing. The application of computers to the control of the machine allows fine precision, reproducibility, and automation. For this project, we created a three-axis machine for the purpose of rendering two dimensional vector graphics.

We had several goals for this project and the machine. The machine would balance high precision and speed, use-limited resources and as many recycled parts as possible, and be reproducible by a hobbyist. The software would correctly transform Scalable Vector Graphics (SVG) image files into G-code machine control instructions, and these instructions would then be used to control the machine. To meet these goals, we used the Inkscape vector image manipulation program to generate SVG files. We then used our modified version of Gcodetools to convert the SVG into Gcode. This G-code was loaded into the Enhanced Machine Controller (EMC), an open source machine control suite. We configured EMC to correctly control our machine. The machine interprets the control signals using a set of custom driver circuits, which drive stepper motors mounted in a frame of our own construction. This results in a highly precise rendering of the input file on paper. All stages of the project were able to meet all of our expectations.

2. Project Description

2.1 Software

2.1.1 Software Overview

Our software system consists of several independent components we have integrated together. To control the machine, we used Enhance Machine Controller (EMC). To generate the G-code used by EMC, we created a patch for the Gcodetools project, an Inkscape plugin. Inkscape was used to generate and manipulate SVG files.

2.1.2 Design Considerations

Engineering as a discipline often requires more integration than large amounts of original development. In a typical project, writing new code presents significant challenges, and the number of features shared between projects means that it is possible to create shared components which implement common features. A library or an existing module allows the use of a well developed and tested component, which saves significant resources in the implementation of the project. The drawback of components is the need to integrate various potentially conflicting interfaces, and the need to understand a complex system in order to effectively use the component.

Components can be purchased, or may be freely available, as in the case of Open Source software. Open Source also provides the opportunity to contribute new features and bug fixes back in to the community. The programs and tools we chose for this project are all open source, and use international standards, which allowed us to rapidly develop the features needed. The open source nature of the

Gcodetools project allowed us to make modifications that improve the product for the entire community.

Originally, in the planning phase, we intended to write both the conversion software and the control software from scratch. Both projects would have been beyond the scope of this project and time allotted. Upon researching available solutions, we came to the conclusion that integration would be a significant challenge, but less costly than creating the code from scratch. Additionally, the Gcodetools project had the requisite functionality but suffered from poor software engineering, and provided an opportunity to make significant contributions to a code base without having to start from nothing.

2.1.3 SVG

Scalable Vector Graphics (SVG) is a World Wide Web Consortium (W3C) standard for describing two dimensional vector image files. Vector images consist of shapes, line vectors and style information instead of the arrays of pixels available in raster images like JPEG or PNG. SVG files are ASCII text documents in XML format, and can be manipulated with a drawing program (such as the open source editor Inkscape) or with a text editor as plain text.

2.1.4 G-code

G-code is an industry standard for a machine control instruction set, specified in several international standards including RS274D and ISO 6983. G-code files are ASCII text files, consisting of a sequence of command codes. Each command code is, in general, a single alphabetical character followed by numeric parameters. While standards exist, many proprietary extensions and modifications are introduced by manufacturers for their specific machines. The G-code produced by the Gcodetools conversion software conforms to the standard expected by Enhanced Machine Controller.

2.1.5 Gcodetools

Gcodetools is an open source Inkscape extension, written in the Python programming language. Inkscape extensions work in the standard Unix IO model, taking SVG on standard input, and output transformed SVG on standard output. The Gcodetools extension generates G-Code from the SVG input and writes it to a file as a side effect of the SVG transformation. The user is able to execute the extension through a configuration dialog. Configuration dialogs are implemented using Inkscape’s user interface descriptor XML files, which describe the layout and types of inputs, which map the input from the user to parameters in the extension script.

While the software from the repository contained all the functionality needed to convert G-Code, the software worked intermittently, failing to function correctly in common cases and crashing in others. The code demonstrated poor software engineering principles, with monolithic classes, poorly formatted code, and many other problems. The pylint code convention analysis tool initially crashed while trying to score the code, and a newer version of pylint gave the code a score of -13.7 out of 10.0. This provided an opportunity to work with an existing code base and improve the functionality. The first step was to implement a proper logging system to replace the custom logging implemented in the module. The standard Python library logging library was used, removing the custom logging logic, and implementing a new logging handler class to maintain the communication with the Inkscape feedback library calls.

The next step was to isolate errors in the code which prevent the execution of the default code configuration. We were able to isolate and patch the bugs which made it impossible to execute our specific code path. The user interface was overly complicated, with most values not having safe defaults. We created a new user interface to limit the number of variables available to the user, and modified the Python argument parsing to include sane default values.

2.1.6 Enhanced Machine Controller

The G-code instructions are loaded into Enhanced Machine Controller (EMC), an open-source machine control suite. EMC provides a G-code interpreter, a runtime environment and a hardware abstraction layer. The hardware abstraction layer emits simple control signals for motor steps and direction, which are sent to the CNC machine over the parallel port connection. Besides the control signals for each axis, the software also sends an Emergency Stop (ESTOP) signal to kill the machine power and receives input from home and limit switches.

The EMC software is a complicated set of applications, and is highly configurable. The configuration is complex and allows a wide range of capabilities through the hardware abstraction layer. This complexity causes a steep learning curve to getting the software configured to work in the correct fashion with our machine. Configuring EMC is accomplished through a set of configuration text files. Various machine communication pinouts and signal parameters need to be configured, as well as the mapping of axis signals into physical measurements. Additionally, we wanted to be able to do manual jogging using a USB joystick. The hardware abstraction layer provides a component system to create complex control logic. A custom HAL configuration was created to use the input from the joystick to control various parts of the EMC software.

2.2 Hardware

2.2.1 Hardware Overview

Our hardware system consists of a wooden frame, on which is mounted three axis of motion in a standard Cartesian coordinate system. Each axis is driven by a stepper motor driven by a custom motor driver circuit. The control of the stepper motors was the primary challenge of the hardware systems, requiring circuit design, prototyping and fabrication.

2.2.2 Microcontroller prototyping

The Atmel AVR family Atmega328p was selected as the micro-controller for this project. This micro-controller provides a variety of features needed for the project, while being cheap and readily available. The controller has 1Kb of RAM and 32Kb of program flash, 23 general purpose IO lines, as well as a variety of hardware features such as in-system programming and RS-232 and SPI serial communications. The AVR family is well supported by the gcc-avr project and the Eclipse C development environment. Originally, we planned to do all the Gcode interpretation and machine control using the microcontroller. It became clear that this was complex of a task, both for the complexity of the implementation and the limitations of the microcontroller capabilities. Therefore we decided to use the EMC software for our final control solution, and used the microcontroller during the prototyping phase.

We used the Arduino Uno to send step and direction signals to the L297, which drives the L298 H-Bridge chip. We used this method to build the analog circuit and test-drive the motor. During the testing, it became clear that the salvaged motors were not capable of the speed and power necessary to drive the X and Y axes. We then acquired more powerful motors to solve this issue.

2.2.3 Analog Circuit

At first, we tested many solutions for the motor driver circuit including circuits. One circuit used the Allegro A3967 and A4983 integrated circuits (IC), each a combination H-bridge and stepper controller. Another circuit used the SGS-Thomson L297 and L298 ICs, a stepper controller and H-bridge driver respectively. We wanted a circuit that could drive at high current - 2 amps per motor coil - and have enough torque to move the draw surface and the z axis around at a fast speed - 15 inches/minute.

The A3967 is a very cheap and generic stepper controller solution for driving standard stepper motors; however, it is only capable of driving a maximum of 750 mA per coil with a maximum voltage of 30V, with a standard operating current of 150mA per coil. The driver IC alone doesn't provide enough current to move the motor at all.

The A4983 is another stepper controller, which solves the limited current problem of the A3967, as it is capable of providing much higher current, up 2A per coil with a maximum voltage of 35V. However, the surface area of the chip was too small to mount an effective heat sink. The chip does come with a heating solution which automatically shuts the IC off at high temperatures. This is a desirable feature in extreme cases, but the driver overheated too easily at the high current and prolonged use of our application.

The L297 digital IC takes the signals from the controller and translates them into stepping signals to send to the L298. The L298 is an H-bridge circuit, which provides the high current required to drive the stepper motor. The L298 provides a capacity of 2A of current per coil. The L297 is also capable of sensing the amount of current flowing through the coils and smoothing the signal to the L298 chip so that the average current flow is more stable.

The L297/L298 combo is also desirable because it can handle higher operating temperatures. The L298 has a larger surface area for mounting a heat sink, which provides a better cooling solution. The driver circuit can also handle higher voltages – up to 46V - and can be overdriven. Early in the development stage, we tried to overdrive the circuit by changing the current sensing between the two chips. This caused an overheating issue and failed to provide the performance boost that we were hoping for. We ended up using the standard configuration of half step mode.

The bipolar stepper motor configuration fits best for this application. We tested the 3 different circuits, and chose to build our circuits using the L297/L298 configuration. The circuit diagram can be found in Appendix A.

For the Z-axis, a small driver circuit board was purchased, as the power requirements of the smaller motor driving the Z-axis does not require the higher power provided by the L297/L298 circuit.

2.2.4 Stepper Motors

Stepper motors are more precise for a task like routing for CNC machines, which made them a desirable choice for our project. Unlike DC motors, the stepper motors are brushless, synchronous electric motors that can divide a full rotation into a large number of steps. This allows for precise

control without any feedback system depending on the size of the project and application. Stepper motors are constant power devices, as speed increases, the torque decreases. Stepper motors come in different types: uni-polar which are easy to drive but have low torque and speed, and bipolar which are hard to drive but have high torque and high speed.

We tested some salvaged motors from scanners and printers. Several were designed for driving belt systems, others were gear driven. The motors were not powerful enough to drive the X and Y axes, which was discovered during prototyping. We decided to purchase two stronger motors for the X and Y axes. For the Z axis, a salvaged motor proved adequate. The motors we purchased step at 1.8 degrees per full step, and are capable of driving 280 ounce-inches of torque, and can be driven with 2.1A per coil bipolar in series, and 4.2A per coil in parallel. Parallel driving allows the stepper motor to operate at higher speeds with higher torque. It was not necessary to have the motor coils in a parallel configuration for our implementation, as the series configuration provided enough power and speed for our needs.

2.2.5 Emergency Stop Relay

The motor driver circuit would continue to be supplied current when the motor was not moving. Because a single transistor path inside the L298 IC was maintaining a constant current, the IC was overheating. The large heat sink was unable to dissipate the heat this produced. The initial solution was to disable the IC when the motor was not moving, as the current through the IC while it was switching did not cause overheating.

To solve this problem, we introduced a relay between the power supply and the IC, which would only supply power when the relay was active. At the same time, we also added a set of fans to move air across the heat sinks attached to the ICs. The fans alone solved the heating issue, but we repurposed the relay solution as an emergency stop solution to kill the power. Control for the emergency stop was provided by the EMC software.

2.2.6 Positional Feedback

Initially we planned to integrate a positional feedback system to ensure that the distance traveled correlated to the distance implied by step counting. We implemented this system using an infrared LED emitter and collector which were placed in close proximity and separated by a slotted wheel which would be attached to the motor rods. When the wheel spun it would cut off the light entering the collector, which would drop the current through the collector. When we implemented this circuit, we found that the current was not dropping fast enough before the collector was re-energized, making it impossible to register a change in current. We also found out that the EMC software had no support for feedback in the configuration that we used. This feature was removed from the final product but the circuit diagram can be found in Appendix A.

2.3 Machine Construction

2.3.1 Frame

The frame was created from ½ particle board. Since the ACME threaded rod was purchased in three foot lengths, the frame was built as a three foot by three foot square platform. The base of the frame

needed to be stabilized so it would not shift, so wood blocks were placed in the corners and a stabilizing wire, going diagonally from corner to corner, was installed to make it a rigid structure. After several weeks the wire stretched so we replaced the wire with a board. This created a very rigid base which no longer moved.

Figure 1 & 2. Frame Base

The construction of the X axis needed a frame to hold the threaded rod above the Y axis. The frame was held together by L-brackets. Since L-brackets are very stable, we used them as an integral part of the construction of the frame. The corners of the frame had the metal bracket pictured in Figure 2. The X axis was connected to the base using the bracket shown in Figure 3. The final connection is shown in Figure 4.

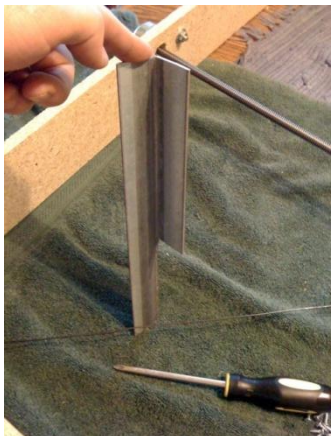


Figure 3 & 4. Assembly Brackets

The Y axis sled is 31 inches wide and is required to slide smoothly in both the negative and positive Y directions. Drawer rails were used to attach the Y axis sled to the frame. These were a very good option because the rails have ball bearings and tight tolerances. The rail that was used in the final implementation allowed for 13 inches of travel in the Y direction. After it was installed in the frame the center of the Y axis sled was unstable, so a stabilizing bar was installed to reduce vibration. The

bushings used on the Y axis stabilizing bar would not slide when a crude harness was placed around the bushing for the purpose of holding it against the Y axis sled. After looking into why this was happening, we learned that bushings cannot be pinched. By pushing on two sided of the bushing would pinch the rod on the inside of the bushing. This prevented the bushing from sliding. A new harness needed to be created that would provide equal pressure around the pushing. After this was created the Y axis sled moved in and out smoothly.

Figure 5 & 6. Y Axis Assembly

2.3.2 X and Y Axes

The X axis consisted of a sled that moved from left to right, holding the Z axis in place. To hold the Z axis steady as it moved from left to right, two stabilizing bars were used. The ½ inch bar was placed on both sided of the acme threaded rod. The Z axis would slide on the bar with fitted brass tubing used as a bushing. This was fastened to the X axis sled with a piece of aluminum. To calibrate the X axis sled to move without binding I loosened the bar so it was floating and then set the sled on one end of the bar. At that end I would fasten the bar in place. This way the bar was perfectly spaced for the bushing to slide on. I then moved the sled over to the other side of the bar and then fastened the bar on the other end. The Z axis moved from side to side by a nut fastened to the back of the Z axis. When the threaded rod was turned the nut pulled the Z axis along.

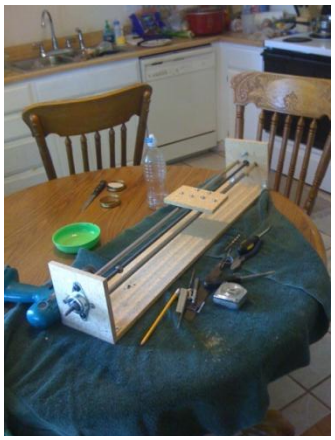


Figure 7, 8, & 9. X & Y Axis Assembly

The construction of the Y axis consisted of a sled that was 13 inches by 31 inches moving forward and backward. The Y axis sled was very large and needed a stabilizing bar in the middle. A 5/8 inch bar was used and two bushings would hold the bar to the sled. A piece of aluminum held the bushing in place so it would not slide around. The acme rod moved the sled with a nut attached to the sled. As the threaded rod would turn the nut would move down the threaded rod pulling the sled with it. To make sure that the bushings and the acme nut was spaced perfectly away from the sled, door shims were used. This made it easier to install the threaded rod and stabilizing bar.

Once the X and Y axis was secured I created the motor housing. This would be constructed of pieces of angle iron put back to back. Two pieces would then be butted up against each other. A hole would then be cut out and the motor would be placed in the hole. The structure was held together with threaded rob and bolts.

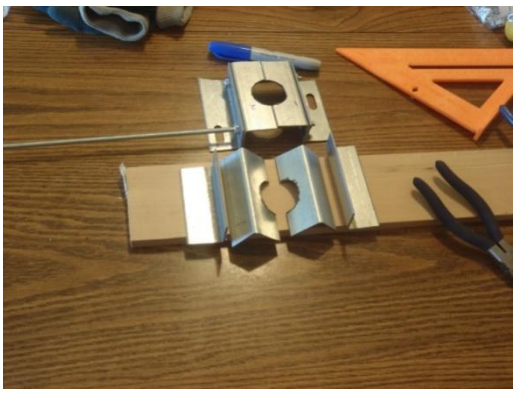


Figure 10 & 11. Motor Mounts

To connect the threaded rod to the motor shaft the threaded rod was filed down to fit the coupling. The first time this was done it was filed by hand. This created a wobble that was not acceptable. This was cut out and the threaded rod was spun with a drill and a file was used to make a true fitting diameter for the coupling. After the coupling was installed the threaded rod had trouble staying connected. Over time the threaded rod would slip. A large divot in the threaded rod and the motor shaft was created and lock-tight was used to prevent the screws from coming off.

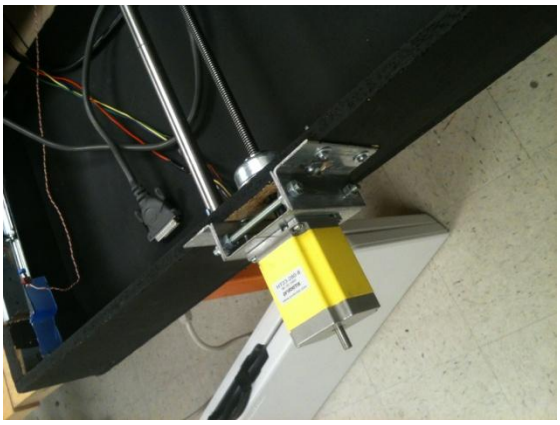


Figure 12. Motor connected to frame

2.3.3 Z Axis

We chose to use a full-step stepper motor that we salvaged from a printer. This was a good fit for the z-axis because the moving parts are relatively small and lightweight and we also drive the z-axis at a lower speed which gives us greater torque from the motor. We used epoxy to attach the motor shaft to a gear found in a computer CD drive. We also mounted the CD drive housing to a board which was attached to the x-axis. We then mounted a pen holder to a drawer slide which we in turn mounted to the CD drive tray which moves up and down when the motor turns the gear. The drawer slide allows our pen to slide up and down in the z-direction while the motor stays in one position in order to compensate for imperfections in the drawing surface on the y-axis.

Figure 13. Z axis

3. Conclusion

We were able to produce a machine that performed well enough to meet our goals for precision, cost, and deadline. We started early on the hardware which allowed us to work through some of the problems we encountered such as needing to upgrade our motors for the x and y-axes and rewiring our motor driver circuits. This reduced the risk associated with hardware prototyping which was by far the highest risk part of our project. We were also able to implement all of the necessary components of our machine a couple of weeks before our project deadline. We had to cut a few of our extra features in order to meet our deadline, such as a positional feedback system and an automatic tool changer, but we were successful because we prioritized these features and worked on the essentials first. Our success can easily be attributed to the planning and prioritizing we did prior to beginning the project. We left ourselves room in our schedule for unexpected delays and began our project early. All of these factors contributed to a successful project which began in the planning phase and ended with a deliverable product which meets the expectations we set for ourselves.

4. Bill of Materials

Item	Vendor	Total Price - USD
Wholesale tools	threaded rod and nut	$22.62
NPS	Drawer and metal rod	$4.92
Drill Spot	Mounted Ball Bearings	$67.44
Marshall's	brass tube	$5.00
HOme Depot	½ partical board	$14.37
NPS	metal rods	$1.62
Walmart	screws	$2.97
Home Depot	Metric Nuts	$0.68
Lowe's	½ rnd steel	$1.26
NPS	Drawer hardware	$2.14
ENCO	hex nuts, coupling	$7.00
ENCO	roll pin	$5.00
ENCO	washers	$4.00
ENCO	washers	$6.00
ENCO	screws	$1.00
ENCO	threaded screw	$6.00
ENCO	screws	$7.00
ENCO	washers	$5.00
ENCO	hex nuts	$5.00
Lowe's	½ steel spacer	$2.53
Marshall's	bearings, couplings	$31.00
NPS	Drawer and metal rod	$3.17
Lowes	Paint	$12.00
Lowes	Paint Supplies	$13.00
Raelco	Wires	$2.78
Newark	Diodes	$20.96
Raelco	Prototype diodes	$31.90
Stock room	Discrete Components	$1.55
Stock room	Wire Spools	$4.40
Probotix	Motors	$118.85
Monoprice	Cable	$3.16
Tayda	L297/L298	$13.23
Sparkfun	Arduino - 2	$79.60
Tayda	L297/L298	$13.62
Tayda	Replacements parts	$14.58
Total Cost:		$535.35

5. References

Advertisement. Home Improvement Made Easy with New Lower Prices | Improve & Repair with The Home Depot. Web. 25 Apr. 2011. <http://www.homedepot.com/>.

"ATmega48A/48PA/88A/88PA/168A/168PA/328/328P Datasheet." Atmel Corporation, Aug. 2010. Web. 25 Apr. 2011. <http://www.atmel.com/dyn/resources/prod_documents/doc8271.pdf>.

Closed, Soldering Sj2. "Easy Driver Stepper Motor Driver." *SchmalzHaus.com Brian Schmalz Homepage*. Web. 08 Apr. 2011. <http://www.schmalzhaus.com/EasyDriver/>.

Cnc Machine Manufacturers. Photograph. *Review about Cnc Machine*. Web. 25 Apr. 2011. <http://www.cncmachine-details.info/cnc-machine-manufacturers.html>.

"Design Fundamentals for Phototransistor Circuits." Fairchild Semiconductor, 30 Apr. 2002. Web. 25 Apr. 2011. <http://www.fairchildsemi.com/an/AN/AN-3005.pdf>.

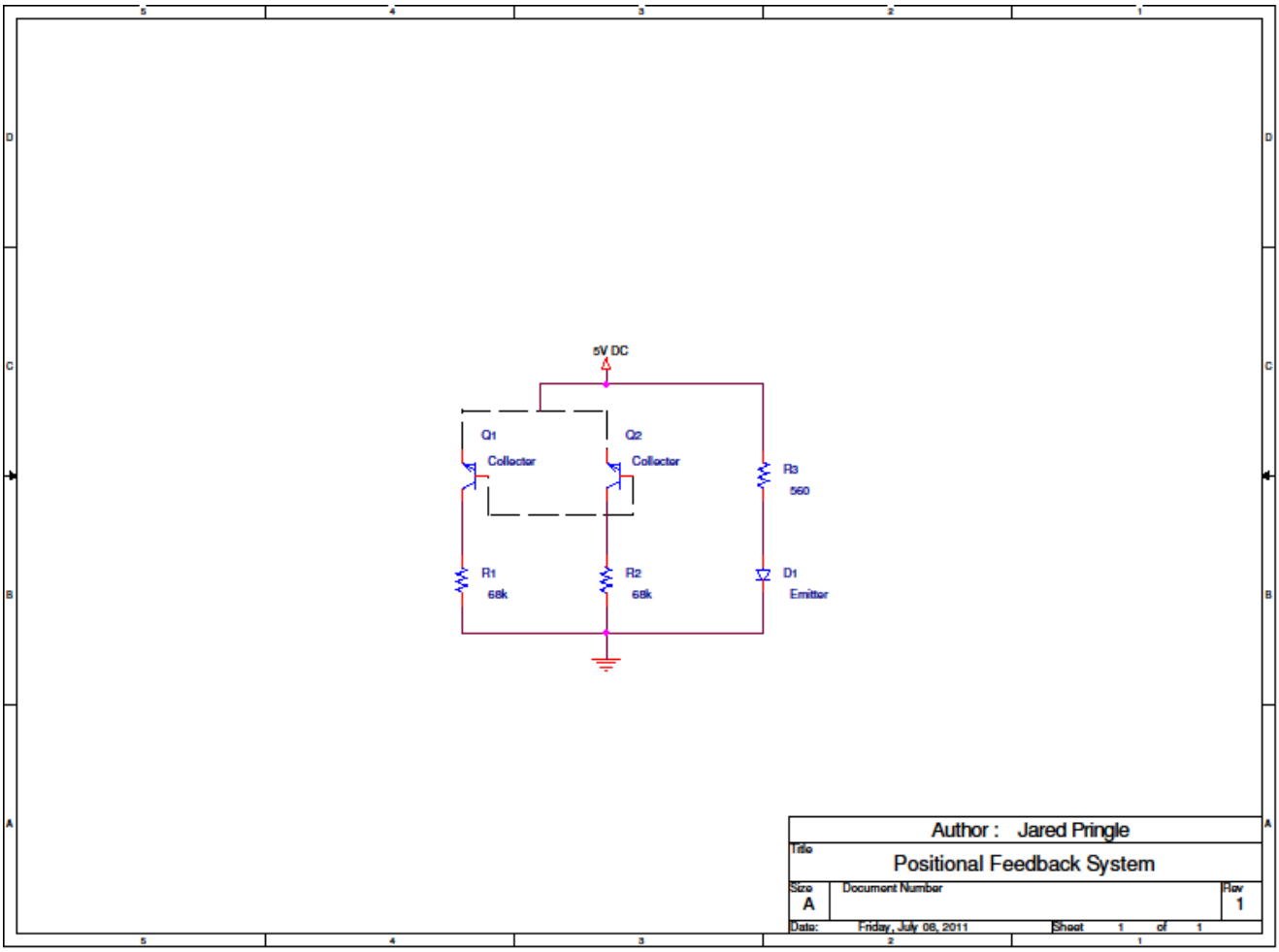
"EMC2 Documentation." Enhanced Machine Controller Project. Web. 25 Apr. 2011. <http://www.linuxcnc.org/docview/html/>.

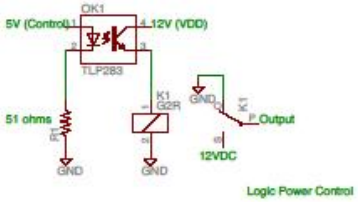
"Inkscape User Documentation." *Inkscape. Draw Freely.* Web. 04 May 2011. <http://inkscape.org/doc/index.php?lang=en>.

"Scalable Vector Graphics (SVG) 1.1 (Second Edition)." World Wide Web Consortium (W3C), 22 June 2010. Web. 25 Apr. 2011. <http://www.w3.org/TR/SVG11/>.

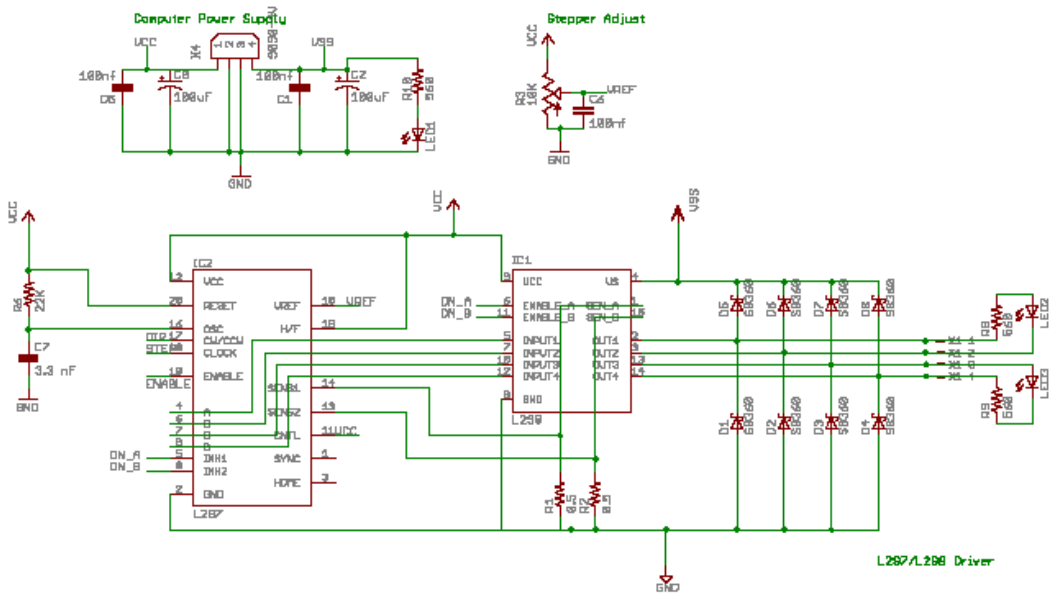
"Standard Cataloged Acme Inch Screw and Nut Quick Reference Chart - Nook Industries, Inc. PowerAc Acme Screws and Nuts." Nook Industries : Linear Actuators for Motion Control Ball Screws, Screw Jacks, Lead Screws, Linear Slides, Acme Screw, Actuator. 25 Apr. 2011. Web. 25 Apr. 2011. <http://www.nookindustries.com/acme/AcmeInchAvailability.cfm>.

Appendix A: Analog Circuit Design



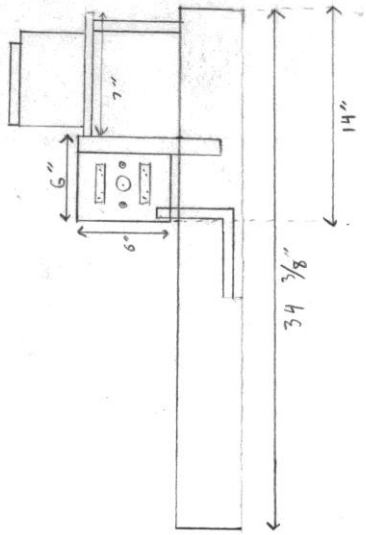


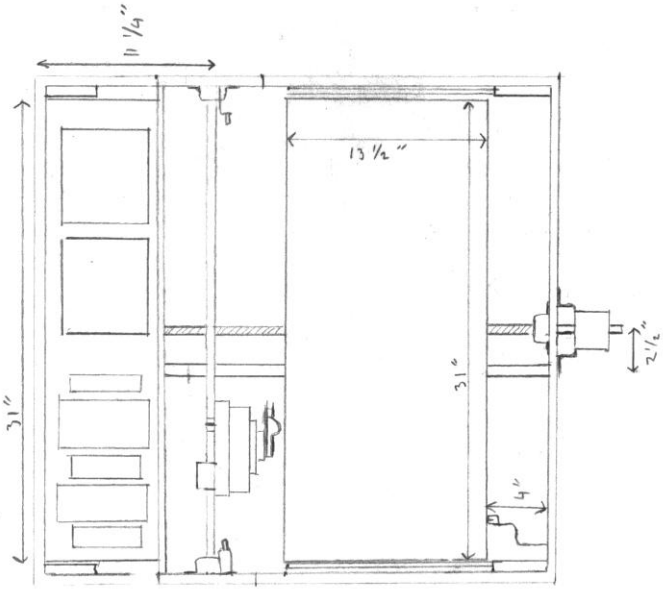
12/3/11 10:22 AM /Users/anhluong/Documents/eagle/StepperDriver/LogicControl12V.sch (Sheet: 1/1)

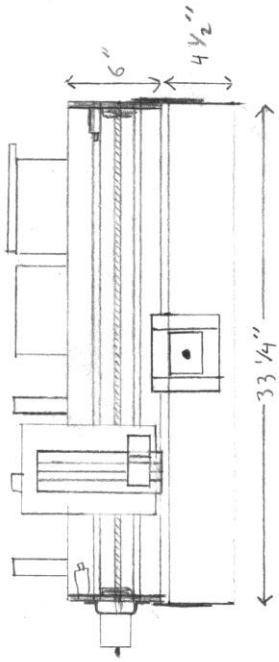


12/3/11 10:20 AM /Users/anhluong/Documents/eagle/StepperDriver/stepper-motor-driver.sch (Sheet: 1/1)

Appendix B: Frame Blueprints







Appendix C: Configuration and Software

C.1 joypad.hal

```
# Manual jogging control for EMC using Logitech Extreme joystick
# Load joystick into thread.
loadusr -W hal_input Extreme

# Painted button mapping                function
#     1 (trigger)input.0.btn-joystick  enable X and Y jog
#     2          input.0.btn-thumb     enable Z jog
#     3          input.0.btn-thumb2    jog speed 9
#     4          input.0.btn-top       jog speed 12
#     5          input.0.btn-top2 jog speed 6
#     6          input.0.btn-pinkie    jog speed 15
#     7          input.0.btn-base halui.program.pause/stop
#     8          input.0.btn-base2     halui.program.resume/run
#     9          input.0.btn-base3     halui.machine.on/off toggle
#     10         input.0.btn-base4     halui.home-all
#     11         input.0.btn-base5     halui.estop.activate
#     12         input.0.btn-base6     halui.estop.reset
#     hat-x      input.0.abs-hat0x-*   digital X jog
#     hat-y      input.0.abs-hat0y-*   digital Y jog

#-------------------------------------------------------------------------------
# Button mapping
#-------------------------------------------------------------------------------
net power_button <= input.0.btn-base3
net estop_restart <= input.0.btn-base5
net estop_activate <= input.0.btn-base6
net home_all <= input.0.btn-base4
net jog-enable-xy <= input.0.btn-joystick
net jog-enable-z <= input.0.btn-thumb
net program_run_pause <= input.0.btn-base2
net program_stop <= input.0.btn-base
net speed_select_0 <= input.0.btn-top2
net speed_select_1 <= input.0.btn-thumb2
net speed_select_2 <= input.0.btn-top
net speed_select_3 <= input.0.btn-pinkie
#net abort_all <= input.0.btn-base2

#-------------------------------------------------------------------------------
# Axis mapping
#-------------------------------------------------------------------------------
net x-position <= input.0.abs-x-position
net y-position <= input.0.abs-y-position
net z-position <= input.0.abs-throttle-position
```

```
net hat_x_counts <= input.0.abs-hat0x-counts
net hat_x_input <= input.0.abs-hat0x-position

net hat_y_counts <= input.0.abs-hat0y-counts
net hat_y_input <= input.0.abs-hat0y-position

# Invert the X and Y axis for correct cartesian direction
setp input.0.abs-x-scale 512
setp input.0.abs-y-scale -512
setp input.0.abs-throttle-scale -127
setp input.0.abs-hat0y-scale -1

#-------------------------------------------------------------------------------
# Load components
#-------------------------------------------------------------------------------
loadrt and2 names=machine-on-and,machine-off-and,run-idle,run-paused,run-
running,hat-x-and-incr,hat-y-and-incr,hat-x-and-decr,hat-y-and-decr
loadrt not names=not-machine-on,not-hat-x-dir-sign,not-hat-y-dir-sign
loadrt conv_s32_bit names=hat-x-non-zero,hat-y-non-zero
loadrt abs names=hat-x-dir,hat-y-dir
loadrt mux2 names=joy-x-enable-mux,joy-y-enable-mux,joy-z-enable-mux
loadrt mux4 names=jog-speed-select
loadrt flipflop names=sel0-flipflop,sel1-flipflop
loadrt or2 names=sel1-set-or,sel1-reset-or,sel0-set-or,sel0-reset-or

addf joy-x-enable-mux servo-thread
addf joy-y-enable-mux servo-thread
addf joy-z-enable-mux servo-thread

addf machine-on-and servo-thread
addf machine-off-and servo-thread
addf not-machine-on servo-thread

addf hat-x-non-zero servo-thread
addf hat-y-non-zero servo-thread
addf hat-x-dir servo-thread
addf hat-y-dir servo-thread
addf hat-x-and-incr servo-thread
addf hat-y-and-incr servo-thread
addf hat-x-and-decr servo-thread
addf hat-y-and-decr servo-thread
addf not-hat-x-dir-sign servo-thread
addf not-hat-y-dir-sign servo-thread

addf jog-speed-select servo-thread
addf sel0-flipflop servo-thread
addf sel1-flipflop servo-thread
addf sel0-set-or servo-thread
addf sel0-reset-or servo-thread
addf sel1-set-or servo-thread
addf sel1-reset-or servo-thread
```

```
addf run-idle servo-thread
addf run-paused servo-thread
addf run-running servo-thread

#-------------------------------------------------------------------------------
# Machine enable operations
#-------------------------------------------------------------------------------
net estop_restart => halui.estop.activate
net estop_activate => halui.estop.reset
#net abort_all => halui.abort
net home_all => halui.home-all

#-------------------------------------------------------------------------------
# Machine toggle power on/off
#-------------------------------------------------------------------------------
# power_button && !halui.machine.is-on => halui.machine.on
net power_button => machine-on-and.in0
net is_machine_on halui.machine.is-on => not-machine-on.in
net not_machine_on not-machine-on.out => machine-on-and.in1
net toggle_machine_on machine-on-and.out => halui.machine.on

# power_button && halui.machine.is-on => halui.machine.off
net power_button => machine-off-and.in0
net is_machine_on halui.machine.is-on => machine-off-and.in1
net toggle_machine_off machine-off-and.out => halui.machine.off

#-------------------------------------------------------------------------------
# Analog jog
#-------------------------------------------------------------------------------
setp joy-x-enable-mux.in0 0.0
net x-position => joy-x-enable-mux.in1
net jog-enable-xy => joy-x-enable-mux.sel
net joy-x-jog joy-x-enable-mux.out => halui.jog.0.analog

setp joy-y-enable-mux.in0 0.0
net y-position => joy-y-enable-mux.in1
net jog-enable-xy => joy-y-enable-mux.sel
net joy-y-jog joy-y-enable-mux.out => halui.jog.1.analog

setp joy-z-enable-mux.in0 0.0
net z-position => joy-z-enable-mux.in1
net jog-enable-z => joy-z-enable-mux.sel

net joy-z-jog joy-z-enable-mux.out => halui.jog.2.analog
setp halui.jog-deadband 0.2

#-------------------------------------------------------------------------------
# Hat decoding
#-------------------------------------------------------------------------------
# position != 0 && positive-dir => halui.jog.X.minus
# position != 0 && negative-dir => halui.jog.X.minus
net hat_x_counts => hat-x-non-zero.in
```

```
net hat_x_input => hat-x-dir.in
net velocity_x_is_negative hat-x-dir.sign => not-hat-x-dir-sign.in

net velocity_x_change <= hat-x-non-zero.out
net velocity_x_change => hat-x-and-decr.in0
net velocity_x_is_negative hat-x-dir.sign => hat-x-and-decr.in1
net velocity_x_change => hat-x-and-incr.in0
net velocity_x_is_positive not-hat-x-dir-sign.out => hat-x-and-incr.in1

net velocity_x_decrease hat-x-and-decr.out => halui.jog.0.minus
net velocity_x_increase hat-x-and-incr.out => halui.jog.0.plus

net hat_y_counts => hat-y-non-zero.in
net hat_y_input => hat-y-dir.in
net velocity_y_is_negative hat-y-dir.sign => not-hat-y-dir-sign.in

net velocity_y_change <= hat-y-non-zero.out
net velocity_y_change => hat-y-and-decr.in0
net velocity_y_is_negative hat-y-dir.sign => hat-y-and-decr.in1
net velocity_y_change => hat-y-and-incr.in0
net velocity_y_is_positive not-hat-y-dir-sign.out => hat-y-and-incr.in1

net velocity_y_decrease hat-y-and-decr.out => halui.jog.1.minus
net velocity_y_increase hat-y-and-incr.out => halui.jog.1.plus

#net velocity_y_decrease hat-y-and-decr.out => halui.max-velocity.decrease
#net velocity_y_increase hat-y-and-incr.out => halui.max-velocity.increase
#setp halui.max-velocity.scale 0.02

#-------------------------------------------------------------------------------
# Set jog speed, 2, 5, 10, 20
#-------------------------------------------------------------------------------
# set selector LSB on (3 || 1), reset on (2 || 0)
net speed_select_0 => sel0-reset-or.in0
net speed_select_2 => sel0-reset-or.in1
net speed_select_1 => sel0-set-or.in0
net speed_select_3 => sel0-set-or.in1
net speed_sel0_reset sel0-reset-or.out => sel0-flipflop.reset
net speed_sel0_set sel0-set-or.out => sel0-flipflop.set

# set selector MSB on (3 || 2), reset on (1 || 0)
net speed_select_0 => sel1-reset-or.in0
net speed_select_1 => sel1-reset-or.in1
net speed_select_2 => sel1-set-or.in0
net speed_select_3 => sel1-set-or.in1
net speed_sel1_reset sel1-reset-or.out => sel1-flipflop.reset
net speed_sel1_set sel1-set-or.out => sel1-flipflop.set

# Select from 4 values
setp jog-speed-select.in0 6
setp jog-speed-select.in1 9
setp jog-speed-select.in2 12
```

```
setp jog-speed-select.in3 15
net joy-speed-1 jog-speed-select.sel0 <= sel0-flipflop.out
net joy-speed-2 jog-speed-select.sel1 <= sel1-flipflop.out
net joy-speed-final halui.jog-speed <= jog-speed-select.out
#net joy-speed-final axis.0.jog-scale <= jog-speed-select.out
#net joy-speed-final axis.1.jog-scale <= jog-speed-select.out
#net joy-speed-final axis.2.jog-scale <= jog-speed-select.out

#-------------------------------------------------------------------------------
# Run, pause, resume
#-------------------------------------------------------------------------------
net program_stop => halui.program.stop
# program_run_pause && halui.program.is-idle => halui.program.run
net program_run_pause => run-idle.in0
net is_idle halui.program.is-idle => run-idle.in1
net is_idle_run run-idle.out => halui.program.run

# program_run_pause && halui.program.is-paused => halui.program.resume
net program_run_pause => run-paused.in0
net is_paused halui.program.is-paused => run-paused.in1
net is_paused_resume run-paused.out => halui.program.resume

# program_run_pause && halui.program.is-running => halui.program.pause
net program_run_pause => run-running.in0
net is_running halui.program.is-running => run-running.in1
net is_running_pause run-running.out => halui.program.pause
```

C.2 teal-mill.ini

```
[EMC]
MACHINE = teal-mill
DEBUG = 0

[DISPLAY]
DISPLAY = axis
EDITOR = gedit
POSITION_OFFSET = RELATIVE
POSITION_FEEDBACK = ACTUAL
MAX_FEED_OVERRIDE = 1.2
INTRO_GRAPHIC = emc2.gif
INTRO_TIME = 5
PROGRAM_PREFIX = /home/teal/emc2/nc_files
INCREMENTS = .1in .05in .01in .005in .001in .0005in .0001in

[FILTER]
PROGRAM_EXTENSION = .png,.gif,.jpg Greyscale Depth Image
PROGRAM_EXTENSION = .py Python Script
png = image-to-gcode
gif = image-to-gcode
jpg = image-to-gcode
py = python
```

```
[TASK]
TASK = milltask
CYCLE_TIME = 0.010

[RS274NGC]
PARAMETER_FILE = emc.var

[EMCMOT]
EMCMOT = motmod
COMM_TIMEOUT = 1.0
COMM_WAIT = 0.010
BASE_PERIOD = 100000
SERVO_PERIOD = 1000000

[HAL]
HALFILE = teal-mill.hal
HALFILE = joypadv3.hal
HALUI = halui

[TRAJ]
AXES = 3
COORDINATES = X Y Z
LINEAR_UNITS = inch
ANGULAR_UNITS = degree
CYCLE_TIME = 0.010
DEFAULT_VELOCITY = 0.10
MAX_LINEAR_VELOCITY = 0.22

[EMCIO]
EMCIO = io
CYCLE_TIME = 0.100
TOOL_TABLE = tool.tbl

[AXIS_0]
TYPE = LINEAR
HOME = 0.0
MAX_VELOCITY = 0.218
MAX_ACCELERATION = 20.0
STEPGEN_MAXACCEL = 25.0
SCALE = 4800.0
FERROR = 0.05
MIN_FERROR = 0.01
MIN_LIMIT = -0.01
MAX_LIMIT = 20.0
HOME_OFFSET = 0.000000
HOME_SEARCH_VEL = 0.218000
HOME_LATCH_VEL = -0.104167
HOME_SEQUENCE = 0

[AXIS_1]
TYPE = LINEAR
```

```
HOME = 0.0
MAX_VELOCITY = 0.218
MAX_ACCELERATION = 20.0
STEPGEN_MAXACCEL = 25.0
SCALE = 4800.0
FERROR = 0.05
MIN_FERROR = 0.01
MIN_LIMIT = -0.01
MAX_LIMIT = 10.0
HOME_OFFSET = 0.000000
HOME_SEARCH_VEL = 0.218000
HOME_LATCH_VEL = 0.104167
HOME_SEQUENCE = 1

[AXIS_2]
TYPE = LINEAR
HOME = 0.0
MAX_VELOCITY = 0.218
MAX_ACCELERATION = 3.0
STEPGEN_MAXACCEL = 3.75
SCALE = 2400.0
FERROR = 0.05
MIN_FERROR = 0.01
MIN_LIMIT = -0.01
MAX_LIMIT = 2.0
HOME_OFFSET = 0.000000
HOME_SEARCH_VEL = 0.150000
HOME_LATCH_VEL = 0.150000
HOME_SEQUENCE = 2
```

C.3 teal-mill.hal

```
loadrt trivkins
loadrt [EMCMOT]EMCMOT base_period_nsec=[EMCMOT]BASE_PERIOD
servo_period_nsec=[EMCMOT]SERVO_PERIOD num_joints=[TRAJ]AXES
loadrt probe_parport
loadrt hal_parport cfg="0x378 out  "
setp parport.0.reset-time 750
loadrt stepgen step_type=0,0,0

addf parport.0.read base-thread
addf stepgen.make-pulses base-thread
addf parport.0.write base-thread
addf parport.0.reset base-thread

addf stepgen.capture-position servo-thread
addf motion-command-handler servo-thread
addf motion-controller servo-thread
addf stepgen.update-freq servo-thread
net spindle-cmd <= motion.spindle-speed-out
```

```
net estop-out => parport.0.pin-01-out
net xstep => parport.0.pin-02-out
setp parport.0.pin-02-out-reset 1
net xdir => parport.0.pin-03-out
net ystep => parport.0.pin-04-out
setp parport.0.pin-04-out-reset 1
setp parport.0.pin-05-out-invert 1
net ydir => parport.0.pin-05-out
net zstep => parport.0.pin-06-out
setp parport.0.pin-06-out-reset 1
net zdir => parport.0.pin-07-out
net astep => parport.0.pin-08-out
setp parport.0.pin-08-out-reset 1
net adir => parport.0.pin-09-out
net estop-out => parport.0.pin-14-out


net home-x <= parport.0.pin-10-in-not
net home-y <= parport.0.pin-11-in
net home-z <= parport.0.pin-12-in-not
net max-x <= parport.0.pin-13-in-not
net max-y <= parport.0.pin-15-in-not


setp stepgen.0.position-scale [AXIS_0]SCALE
setp stepgen.0.steplen 1
setp stepgen.0.stepspace 0
setp stepgen.0.dirhold 31000
setp stepgen.0.dirsetup 26500
setp stepgen.0.maxaccel [AXIS_0]STEPGEN_MAXACCEL
net xpos-cmd axis.0.motor-pos-cmd => stepgen.0.position-cmd
net xpos-fb stepgen.0.position-fb => axis.0.motor-pos-fb
net xstep <= stepgen.0.step
net xdir <= stepgen.0.dir
net xenable axis.0.amp-enable-out => stepgen.0.enable
net home-x => axis.0.home-sw-in
net max-x => axis.0.pos-lim-sw-in

setp stepgen.1.position-scale [AXIS_1]SCALE
setp stepgen.1.steplen 1
setp stepgen.1.stepspace 0
setp stepgen.1.dirhold 31000
setp stepgen.1.dirsetup 26500
setp stepgen.1.maxaccel [AXIS_1]STEPGEN_MAXACCEL
net ypos-cmd axis.1.motor-pos-cmd => stepgen.1.position-cmd
net ypos-fb stepgen.1.position-fb => axis.1.motor-pos-fb
net ystep <= stepgen.1.step
net ydir <= stepgen.1.dir
net yenable axis.1.amp-enable-out => stepgen.1.enable
net home-y => axis.1.home-sw-in
net max-y => axis.1.pos-lim-sw-in
```

```
setp stepgen.2.position-scale [AXIS_2]SCALE
setp stepgen.2.steplen 1
setp stepgen.2.stepspace 0
setp stepgen.2.dirhold 31000
setp stepgen.2.dirsetup 26500
setp stepgen.2.maxaccel [AXIS_2]STEPGEN_MAXACCEL
net zpos-cmd axis.2.motor-pos-cmd => stepgen.2.position-cmd
net zpos-fb stepgen.2.position-fb => axis.2.motor-pos-fb
net zstep <= stepgen.2.step
net zdir <= stepgen.2.dir
net zenable axis.2.amp-enable-out => stepgen.2.enable
net home-z => axis.2.home-sw-in

net estop-out <= iocontrol.0.user-enable-out
net estop-out => iocontrol.0.emc-enable-in

loadusr -W hal_manualtoolchange
net tool-change iocontrol.0.tool-change => hal_manualtoolchange.change
net tool-changed iocontrol.0.tool-changed <= hal_manualtoolchange.changed
net tool-number iocontrol.0.tool-prep-number => hal_manualtoolchange.number
net tool-prepare-loopback iocontrol.0.tool-prepare => iocontrol.0.tool-prepared
```

C.4 Arduino Servo Prototype Sketch

```
// Will Lutz  9-7-2011
// This is two programs together.  One controles the servo for the
// robotic claw.  the other program is for the stepper motor.


// Controlling a servo position using a potentiometer (variable
resistor)
// by Michal Rinott <http://people.interaction-ivrea.it/m.rinott>


#define z_stepPin 4
#define z_dirPin 5
#define x_stepPin 2
#define x_dirPin 3
#define y_stepPin 10
#define y_dirPin 11
#include <Servo.h>


Servo myservo;  // create servo object to control a servo
```

```
int potpin = 0;  // analog pin used to connect the potentiometer
int val;    // variable to read the value from the analog pin

void setup() {
  myservo.attach(9);  // attaches the servo on pin 9 to the servo
object
  Serial.begin(9600);
  Serial.println("Starting stepper exerciser.");

  pinMode(z_stepPin, OUTPUT);
  pinMode(z_dirPin, OUTPUT);
  pinMode(y_stepPin, OUTPUT);
  pinMode(y_dirPin, OUTPUT);
  pinMode(x_stepPin, OUTPUT);
  pinMode(x_dirPin, OUTPUT);

  digitalWrite(z_dirPin, HIGH);
  digitalWrite(z_stepPin, LOW);
  digitalWrite(y_dirPin, HIGH);
  digitalWrite(y_stepPin, LOW);
  digitalWrite(x_dirPin, HIGH);
  digitalWrite(x_stepPin, LOW);

}

void loop() {
  int i, j;



  i = 450;

  Serial.print("Speed: ");
  Serial.println(i);
```

```
  for (j=0;; j++) {
    digitalWrite(z_stepPin, HIGH);
    digitalWrite(y_stepPin, HIGH);
    digitalWrite(x_stepPin, HIGH);
    delayMicroseconds(i);
    digitalWrite(z_stepPin, LOW);
    digitalWrite(y_stepPin, LOW);
    digitalWrite(x_stepPin, LOW);
    delayMicroseconds(i);
    val = analogRead(potpin);            // reads the value of the
potentiometer (value between 0 and 1023)
    val = map(val, 0, 1023, 0, 179);     // scale it to use it with
the servo (value between 0 and 180)
    myservo.write(val);                  // sets the servo position
according to the scaled value
    //delay(1);// was 15                           // waits for the
servo to get there
  }



}
```

C.5 Patch for Gcodetools, branched off revision 209.

The branch for the gcodetools project is located at:

https://code.launchpad.net/~yashton/gcodetools/teal-refactor

```
=== modified file 'gcodetools-dev.inx'
--- gcodetools-dev.inx 2011-06-28 12:55:31 +0000
+++ gcodetools-dev.inx 2011-12-06 04:50:12 +0000
@@ -48,7 +48,7 @@
                <param name="add-numeric-suffix-to-filename" type="boolean"
_gui-text="Add numeric suffix to filename">true</param>

                <param name="directory" type="string" _gui-
text="Directory:">/home</param>
-
+               <param name="load-in-emc" type="boolean" _gui-text="Load
output into AXIS">True</param>
```

```
                 <param name="Zsafe" type="float" precision="5" min="-1000"
max="1000" _gui-text="Z safe height for G00 move over blank:">5</param>
                 <param name="unit" type="enum" _gui-text="Units (mm or in):">
                      <_item value="G21 (All units in mm)">mm</_item>

=== modified file 'gcodetools-dev.py'
--- gcodetools-dev.py  2011-07-02 07:46:08 +0000
+++ gcodetools-dev.py  2011-12-06 04:50:12 +0000
@@ -83,12 +83,24 @@
 import random
 import gettext
 _ = gettext.gettext
-
-
+import logging
+import subprocess
+
+LOG_LEVEL = logging.WARNING
+LOG_FORMAT = '%(asctime)s %(levelname)s %(filename)s:%(lineno)d in
%(funcName)s : %(message)s'
+
+class InkexHandler (logging.Handler):
+     def handle(self, record):
+          inkex.errormsg(self.format(record))
+
+logger = logging.getLogger('gcodetools')
+
 ### Check if inkex has errormsg (0.46 version does not have one.) Could be
removed later.
-if "errormsg" not in dir(inkex):
-     inkex.errormsg = lambda msg: sys.stderr.write((unicode(msg) +
"\n").encode("UTF-8"))
+if "errormsg" in dir(inkex):
+     inkex_handler = InkexHandler()
+     logger.addHandler(inkex_handler)

+logger.setLevel(LOG_LEVEL)

 def bezierslopeatt(((bx0,by0),(bx1,by1),(bx2,by2),(bx3,by3)),t):

      ax,ay,bx,by,cx,cy,x0,y0=bezmisc.bezierparameterize(((bx0,by0),(bx1,by1),(
bx2,by2),(bx3,by3)))
@@ -101,8 +113,8 @@
                 dx = 6*ax
                 dy = 6*ay
                 if dx==dy==0 :
-                      print_("Slope error x = %s*t^3+%s*t^2+%s*t+%s, y =
%s*t^3+%s*t^2+%s*t+%s,  t = %s, dx==dy==0" % (ax,bx,cx,dx,ay,by,cy,dy,t))
-                      print_(((bx0,by0),(bx1,by1),(bx2,by2),(bx3,by3)))
+                      logger.info("Slope error x = %s*t^3+%s*t^2+%s*t+%s, y =
%s*t^3+%s*t^2+%s*t+%s,  t = %s, dx==dy==0" % (ax,bx,cx,dx,ay,by,cy,dy,t))
```

```
+                       logger.info("(%f, %f)"*4,
bx0,by0,bx1,by1,bx2,by2,bx3,by3)
                        dx, dy = 1, 1

      return dx,dy
@@ -349,7 +361,7 @@
                  min_, max_ = line_to_line_min_max_distance_2(points1[i-1],
points1[i], points2[j-1], points2[j])
                  min_dist = min(min_dist,min_)
                  max_dist = max(max_dist,max_)
-                 print_("bound_to_bound", min_dist, max_dist)
+                 logger.info("bound_to_bound %s %s", min_dist, max_dist)
      return min_dist, max_dist

 def csp_to_point_distance(csp, p, dist_bounds = [0,1e100], tolerance=.01) :
@@ -1617,19 +1629,6 @@



################################################################################
#
-###       print_ prints any arguments into specified log file
-
################################################################################
#
-
-def print_(*arg):
-     f = open(options.log_filename,"a")
-     for s in arg :
-          s = str(unicode(s).encode('unicode_escape'))+" "
-          f.write( s )
-     f.write("\n")
-     f.close()
-
-
-
################################################################################
#
 ###       Point (x,y) operations

################################################################################
#
 class P:
@@ -1785,7 +1784,7 @@
                        t1 = ( -v1.x*(b.end.y-self.end.y) + v1.y*(b.end.x-
self.end.x) ) / x
                        t2 = ( -v1.y*(self.st.x-b.st.x) + v1.x*(self.st.y-
b.st.y) ) / x

-                       gcodetools.error((x,t1,t2), "warning")
+                       logger.warning("%f %f %f", x,t1,t2)
                        if 0<=t1<=1 and 0<=t2<=1 : return [ self.st+v1*t1 ]
```

```
                        else : return []
            else: return []
@@ -1869,7 +1868,7 @@
            #Crve defenitnion [start point, type = {'arc','line','move','end'},
arc center, arc angle, end point, [zstart, zend]]
            self.items = []
            for sp in curve:
-                print_(sp)
+                logger.info(str(sp))
                 if sp[1] == 'move':
                      self.items.append([])
                 if sp[1] == 'arc':
@@ -1895,8 +1894,8 @@
      offset_subdivision_depth = 10
      time_ = time.time()
      time_start  = time_
-     print_("Offset start at %s"% time_)
-     print_("Offset radius %s"% r)
+     logger.info("Offset start at %s"% time_)
+     logger.info("Offset radius %s"% r)


      def csp_offset_segment(sp1,sp2,r) :
@@ -2025,7 +2024,7 @@
                        )

            if  err>tolerance**2 and depth>0:
-                #print_(csp_seg_to_point_distance(sp1_r,sp2_r,
(P(csp_at_t(sp1,sp2,.25)) +
P(csp_normalized_normal(sp1,sp2,.25))*r).to_list())[0], tolerance)
+                #logger.info(str(csp_seg_to_point_distance(sp1_r,sp2_r,
(P(csp_at_t(sp1,sp2,.25)) +
P(csp_normalized_normal(sp1,sp2,.25))*r).to_list())[0], tolerance))
                 if depth > offset_subdivision_depth-2 :
                      t = csp_max_curvature(sp1,sp2)
                      t = max(.1,min(.9 ,t))
@@ -2072,8 +2071,8 @@
      original_csp = csp[:]
      # Clip segments which has curvature>1/r. Because their offset will be
selfintersecting and very nasty.

-     print_("Offset prepared the path in %s"%(time.time()-time_))
-     print_("Path length = %s"% sum([len(i)for i in csp] ) )
+     logger.info("Offset prepared the path in %s"%(time.time()-time_))
+     logger.info("Path length = %s"% sum([len(i)for i in csp] ) )
      time_ = time.time()


      ######################################################################
###
@@ -2118,7 +2117,7 @@
```

```
            #     draw_pointer(csp_at_t(subpath_offset[k-1], subpath_offset[k],
t))

      #inkex.etree.SubElement( options.doc_root, inkex.addNS('path','svg'),
{"d": cubicsuperpath.formatPath(unclipped_offset),
"style":"fill:none;stroke:#0f0;"} )
-     print_("Offsetted path in %s"%(time.time()-time_))
+     logger.info("Offsetted path in %s"%(time.time()-time_))
      time_ = time.time()

      #for i in range(len(unclipped_offset)):
@@ -2164,15 +2163,15 @@
                                                intersection[subpath_i] += [
[i,t[0]] ]
                                                intersection[subpath_j] += [
[j,t[1]] ]
                                                #draw_pointer(csp_at_t(subpath[i-
1],subpath[i],t[0]),"#f00")
-                                               #print_(t)
-                                               #print_(i,j)
+                                               #logger.info("%f",t)
+                                               #logger.info("%f %f",i,j)
                                          elif len(t)==5 and t[4]=="Overlap":
                                                intersection[subpath_i] += [
[i,t[0]], [i,t[1]] ]
                                                intersection[subpath_j] += [
[j,t[1]], [j,t[3]] ]

-     print_("Intersections found in %s"%(time.time()-time_))
-     print_("Examined %s segments"%(summ))
-     print_("found %s intersections"%(summ1))
+     logger.info("Intersections found in %s"%(time.time()-time_))
+     logger.info("Examined %s segments"%summ)
+     logger.info("found %s intersections"%summ1)
      time_ = time.time()

      ######################################################################
@@ -2195,7 +2194,7 @@

      #for i in range(len(splitted_offset)):
      #     draw_csp([splitted_offset[i]], color = ["Green","Red","Blue"][i%3])
-     print_("Splitted in %s"%(time.time()-time_))
+     logger.info("Splitted in %s"%(time.time()-time_))
      time_ = time.time()


@@ -2245,7 +2244,7 @@
                 minx,miny,maxx,maxy = csp_true_bounds([s])
                 if (minx[0]-maxx[0])**2 + (miny[1]-maxy[1])**2 < 0.1 :
                       joined_result.remove(s)
-     print_("Clipped and joined path in %s"%(time.time()-time_))
+     logger.info("Clipped and joined path in %s"%(time.time()-time_))
```

```
      time_ = time.time()

      ######################################################################
@@ -2262,9 +2261,9 @@
                       draw_csp([s], comment = math.sqrt(dist[0]))
                       draw_pointer(csp_at_t(csp[dist[1]][dist[2]-
1],csp[dist[1]][dist[2]],dist[3])+s[int(len(s)/2)][1],"blue", "line", comment =
[math.sqrt(dist[0]),i,j,sp]  )

-     print_("-----------------------------")
-     print_("Total offset time %s"%(time.time()-time_start))
-     print_()
+     logger.info("-------------------------------------------------------------
----")
+     logger.info("Total offset time %s"%(time.time()-time_start))
+     logger.info()
      return joined_result


@@ -2431,8 +2430,7 @@


 class Postprocessor():
-     def __init__(self, error_function_handler):
-          self.error = error_function_handler
+     def __init__(self):
           self.functions = {
                            "remap"          : self.remap,
                            "remapi"   : self.remapi ,
@@ -2461,13 +2459,13 @@
      def parse_command(self,command):
           r = re.match(r"([A-Za-z0-9_]+)\s*\(\s*(.*)\)",command)
           if not r:
-                self.error("Parse error while postprocessing.\n(Command:
'%s')"%(command), "error")
+                logger.error("Parse error while postprocessing.\n(Command:
'%s')", command)
           function, parameters = r.group(1).lower(),r.group(2)
           if function in self.functions :
-                print_("Postprocessor: executing function
%s(%s)"%(function,parameters))
+                logger.info("Postprocessor: executing function
%s(%s)"%(function,parameters))
                self.functions[function](parameters)
           else :
-                self.error("Unrecognized function '%s' while
postprocessing.\n(Command: '%s')"%(function,command), "error")
+                logger.error("Unrecognized function '%s' while
postprocessing.\n(Command: '%s')", function, command)


      def re_sub_on_gcode_lines(self, parameters):
```

```
@@ -2478,7 +2476,7 @@
                        self.gcode += eval( "re.sub(%s,line)"%parameters) +"\n"

           except Exception as ex :
-                self.error("Bad parameters for regexp. They should be as
re.sub pattern and replacement parameters! For example: r\"G0(\d)\", r\"G\\1\"
\n(Parameters: '%s')\n %s"%(parameters, ex), "error")
+                logger.error("Bad parameters for regexp. They should be as
re.sub pattern and replacement parameters! For example: r\"G0(\d)\", r\"G\\1\"
\n(Parameters: '%s')\n %s", parameters, ex)


     def remapi(self,parameters):
@@ -2494,7 +2492,7 @@
                s = s.replace(":#:#:coma:#:#:","\,")
                r = re.match("""\s*(\'|\")(.*)\\1\s*-
>\s*(\'|\")(.*)\\3\s*""",s)
                if not r :
-                     self.error("Bad parameters for remap.\n(Parameters:
'%s')"%(parameters), "error")
+                     logger.error("Bad parameters for remap.\n(Parameters:
'%s')", parameters)
                pattern +=[r.group(2)]
                remap +=[r.group(4)]

@@ -2530,9 +2528,12 @@
                if plane not in warned:
                      r = re.search(r"(?i)(G02|G03)", s_wo_comments)
                      if r :
-                           if plane == "g17" and scale[0]!=scale[1]:
self.error("Post-processor: Scale factors for X and Y axis are not the same.
G02 and G03 codes will be corrupted.","warning")
-                           if plane == "g18" and scale[0]!=scale[2]:
self.error("Post-processor: Scale factors for X and Z axis are not the same.
G02 and G03 codes will be corrupted.","warning")
-                           if plane == "g19" and scale[1]!=scale[2]:
self.error("Post-processor: Scale factors for Y and Z axis are not the same.
G02 and G03 codes will be corrupted.","warning")
+                           if plane == "g17" and scale[0]!=scale[1]:
+                                 logger.warning("Post-processor: Scale factors
for X and Y axis are not the same. G02 and G03 codes will be corrupted.")
+                           if plane == "g18" and scale[0]!=scale[2]:
+                                 logger.warning("Post-processor: Scale factors
for X and Z axis are not the same. G02 and G03 codes will be corrupted.")
+                           if plane == "g19" and scale[1]!=scale[2]:
+                                 logger.warning("Post-processor: Scale factors
for Y and Z axis are not the same. G02 and G03 codes will be corrupted.")
                           warned += [plane]
                # Transform
                for i in range(len(axis)) :
@@ -2644,7 +2645,7 @@
           try:
```

```
                round_ = int(parameters)
            except :
-                   self.error("Bad parameters for round. Round should be an
integer! \n(Parameters: '%s')"%(parameters), "error")
+                   logger.error("Bad parameters for round. Round should be an
integer! \n(Parameters: '%s')", parameters)
            gcode = ""
            for s in self.gcode.split("\n"):
                for a in "xyzijkaf" :
@@ -2666,10 +2667,10 @@
            try :
                for i in range(len(parameters)) :
                    if float(parameters[i])==0 :
-                       self.error("Bad parameters for scale. Scale should
not be 0 at any axis! \n(Parameters: '%s')"%(parameters), "error")
+                       logger.error("Bad parameters for scale. Scale
should not be 0 at any axis! \n(Parameters: '%s')", parameters)
                    scale[i] = float(parameters[i])
            except :
-               self.error("Bad parameters for scale.\n(Parameters:
'%s')"%(parameters), "error")
+               logger.error("Bad parameters for scale.\n(Parameters: '%s')",
parameters)
            self.transform([0,0,0,0],scale)


@@ -2680,7 +2681,7 @@
                for i in range(len(parameters)) :
                    move[i] = float(parameters[i])
            except :
-               self.error("Bad parameters for move.\n(Parameters:
'%s')"%(parameters), "error")
+               logger.error("Bad parameters for move.\n(Parameters: '%s')",
parameters)
            self.transform(move,[1.,1.,1.,1.])


@@ -2690,7 +2691,7 @@
            for p in parameters:
                if p in [",","\t"," ","    ","\r","'",'"'] : continue
                if p not in ["x","y","z","a"] :
-                   self.error("Bad parameters for flip_axis. Parameter
should be string consists of 'xyza' \n(Parameters: '%s')"%(parameters),
"error")
+                   logger.error("Bad parameters for flip_axis. Parameter
should be string consists of 'xyza' \n(Parameters: '%s')", parameters)
                axis[p] = -axis[p]
            self.scale("%f,%f,%f,%f"%(axis["x"],axis["y"],axis["z"],axis["a"]))

@@ -2816,7 +2817,7 @@
                                        if dist > d  : dist = d
```

```
            if zerro_plane and dist > 10 + top : dist = 10 + top
-           #print_(dist, top, bottom)
+           #logger.info("%f %f %f", dist, top, bottom)
            #self.draw()
            self.move(0, -dist)

@@ -2842,7 +2843,7 @@
                        if p==st or p==end : return True # point is a vertex =
point is on the edge
                        if st[0]>end[0] : st, end = end, st # This will be
needed to check that edge if open only at rigth end
                        c = (p[1]-st[1])*(end[0]-st[0])-(end[1]-st[1])*(p[0]-
st[0])
-                       #print_(c)
+                       #logger.info(str(c))
                        if st[0]<=p[0]<end[0] :
                              if c<0 :
                                    inside = not inside
@@ -2954,17 +2955,17 @@
                  last_edge = [(last[0][0]-last[1][0])/last[2], (last[0][1]-
last[1][1])/last[2]]
                  for p in edges:

      #draw_pointer(list(p[0])+[p[0][0]+last_edge[0]*40,p[0][1]+last_edge[1]*40
], "Red", "line", width=1)
-                       #print_("len(edges)=",len(edges))
+                       #logger.info("len(edges)= %d",len(edges))
                        cur = [(p[1][0]-p[0][0])/p[2],(p[1][1]-p[0][1])/p[2]]
                        cos, sin = dot(cur,last_edge),  cross(cur,last_edge)

      #draw_pointer(list(p[0])+[p[0][0]+cur[0]*40,p[0][1]+cur[1]*40], "Orange",
"line", width=1, comment = [sin,cos])
-                       #print_("cos, sin=",cos,sin)
-                       #print_("min_angle_before=",min_angle)
+                       #logger.info("cos, sin= %f %f",cos,sin)
+                       #logger.info("min_angle_before= %f",min_angle)

                        if    angle_is_less(sin,cos,min_angle[0],min_angle[1]) :
                              min_angle = [sin,cos]
                              next = p
-                       #print_("min_angle=",min_angle)
+                       #logger.info("min_angle= %f",min_angle)

                  return next

@@ -2990,7 +2991,7 @@
                        loops1 += 1
                        next = get_closes_edge_by_angle(edges[last[1]],last)
                        #draw_pointer(next[0]+next[1],"Green","line", comment=i,
width= 1)
-                       #print_(next[0],"-",next[1])
+                       #logger.info("%f, %f",next[0],next[1])
```

```
                      last = next
                      poly += [ list(last[0]) ]
@@ -3064,8 +3065,8 @@
           #          rank += [ [self.population[i][0] / sim if sim>0 else
1e100,i] ]
           #     rank.sort()
           #     res += [  copy.deepcopy(self.population[rank[0][1]]) ]
-          #     print_(rank[0],self.population[rank[0][1]][0])
-          #     print_(res[-1])
+          #     logger.info(str(rank[0]) +
str(self.population[rank[0][1]][0]))
+          #     logger.info(str(res[-1]))
           #     del self.population[rank[0][1]]

           self.population = res
@@ -3094,7 +3095,8 @@
                      self.incest_mutation_count_multiplyer = 2.
                else :
                      pass
-#                     if random.random()<.01 :
print_(self.species_distance2(parent1, parent2))
+#                     if random.random()<.01 :
+#                           logger.info(str(self.species_distance2(parent1,
parent2)))
                start_gene = random.randint(0,self.genes_count)
                end_gene =
(max(1,random.randint(0,self.genes_count),int(self.genes_count/4))+start_gene)
% self.genes_count
                if end_gene<start_gene :
@@ -3181,7 +3183,7 @@
                      from scipy import weave
                      from scipy.weave import converters
                except:
-                     options.self.error("For this function Scipy is needed.
See http://www.cnc-club.ru/gcodetools for details.","error")
+                     logger.error("Failure to import scipy")

           # Prepare vars
           poly_, subpoly_, points_ = [], [], []
@@ -3221,7 +3223,8 @@
                                             compiler='gcc',
                                             support_code = functions,
                                             )
-          if s!='' : options.self.error(s,"warning")
+          if s != '':
+               logger.warning("%s", s)
           sys.stdout = stdout_

           for i in range(len(test_)):
@@ -3613,21 +3616,34 @@
```

```
     def export_gcode(self,gcode, no_headers = False) :
+          postprocessor = None
           if self.options.postprocessor != ""  or
self.options.postprocessor_custom != "" :
-                postprocessor = Postprocessor(self.error)
+                postprocessor = Postprocessor()
                 postprocessor.gcode = gcode
                 if self.options.postprocessor != "" :
                      postprocessor.process(self.options.postprocessor)
                 if self.options.postprocessor_custom != "" :
                      postprocessor.process(self.options.postprocessor_custom)

-          if not no_headers :
+          if not no_headers and postprocessor is not None:
                 postprocessor.gcode = self.header + postprocessor.gcode +
self.footer
-
-          f = open(self.options.directory+self.options.file, "w")
-          f.write(postprocessor.gcode)
-          f.close()
-
+                output = postprocessor.gcode
+          else:
+                output = gcode
+
+          output_file = self.options.directory + self.options.file
+          logger.info("Writing %d bytes to file %s", len(output),
output_file)
+          f = open(output_file, "w")
+          f.write(output)
+          f.close()
+
+          # Load the output file into EMC2 AXIS
+          if self.options.load_into_emc:
+                status = subprocess.call(["axis-remote", "--ping"])
+                logger.info("Checking status of AXIS: %d", status)
+                if status is 0:
+                      logger.info("Opening file in AXIS")
+                      subprocess.call(["axis-remote", output_file])


##############################################################################
#
 ###       In/out paths:
@@ -3666,7 +3682,7 @@
                      return csp_subpath_line_to([], [sp2[1],p])

           if not self.options.in_out_path and not
self.options.plasma_prepare_corners and
self.options.in_out_path_do_not_add_reference_point:
```

```
-                 self.error("Warning! Extenstion is not said to do anything!
Enable one of Create in-out paths or Prepare corners checkboxes or disable Do
not add in-out referense point!")
+                 logger.warning("Warning! Extenstion is not said to do
anything! Enable one of Create in-out paths or Prepare corners checkboxes or
disable Do not add in-out referense point!")
                  return

            # Add in-out-reference point if there is no one yet.
@@ -3680,14 +3696,14 @@
                  self.set_markers()
                  add_func = {"Round":add_arc, "Perpendicular": add_normal,
"Tangent": add_tangent}[self.options.in_out_path_type]
                  if self.options.in_out_path_type == "Round" and
self.options.in_out_path_len > self.options.in_out_path_radius*3/2*math.pi :
-                       self.error("In-out len is to big for in-out radius will
cropp it to be r*3/2*pi!", "warning")
+                       logger.warning("In-out len is to big for in-out radius
will cropp it to be r*3/2*pi!")

                  if self.selected_paths == {} and
self.options.auto_select_paths:
                        self.selected_paths = self.paths
-                       self.error(_("No paths are selected! Trying to work on
all available paths."),"warning")
+                       logger.warning("No paths are selected! Trying to work on
all available paths.")

                  if self.selected_paths == {}:
-                       self.error(_("Noting is selected. Please select
something."),"warning")
+                       logger.warning("Noting is selected. Please select
something.")
                  a = self.options.plasma_prepare_corners_tolerance
                  corner_tolerance = cross([1.,0.], [math.cos(a),math.sin(a)])

@@ -3791,7 +3807,7 @@
            surface = Polygon()
            polygons = []
            time_ = time.time()
-           print_("Arrangement start at %s"%(time_))
+           logger.info("Arrangement start at %s"%(time_))
            original_paths = []
            for layer in self.layers :
                  if layer in paths :
@@ -3801,13 +3817,13 @@
                              for subpath in csp :
                                    for sp1, sp2 in zip(subpath,subpath[1:]) :

      polygon.add([csp_segment_convex_hull(sp1,sp2)])
-                             #print_("Redused edges count from", sum([len(poly)
for poly in polygon.polygon ]) )
```

```
+                           #logger.info("Redused edges count from %d",
sum([len(poly) for poly in polygon.polygon ]) )
                            polygon.hull()
                            original_paths += [path]
                            polygons += [polygon]

-           print_("Paths hull computed in %s sec."%(time.time()-time_))
-           print_("Got %s polygons having average %s edges each."% (
len(polygons), float(sum([ sum([len(poly) for poly in polygon.polygon]) for
polygon in polygons ])) / len(polygons) ) )
+           logger.info("Paths hull computed in %s sec."%(time.time()-time_))
+           logger.info("Got %s polygons having average %s edges each."% (
len(polygons), float(sum([ sum([len(poly) for poly in polygon.polygon]) for
polygon in polygons ])) / len(polygons) ) )
            time_ = time.time()

 #          material_width = self.options.arrangement_material_width
@@ -3819,7 +3835,7 @@
            population = Arangement_Genetic(polygons, material_width)


-           print_("Genetic algorithm start at %s"%(time_))
+           logger.info("Genetic algorithm start at %s"%(time_))
            start_time = time.time()
            time_ = time.time()

@@ -3827,7 +3843,7 @@

            population.add_random_species(50)
            #population.test(population.test_spiece_centroid)
-           print_("Initial population done in %s"%(time.time()-time_))
+           logger.info("Initial population done in %s"%(time.time()-time_))
            time_ = time.time()
            pop = copy.deepcopy(population)
            population_count = self.options.arrangement_population_count
@@ -3845,7 +3861,7 @@
                  population.move_mutation_factor = 1.
                  population.mutation_genes_count = [1,2]
                  population.populate_species(250, 20)
-                 print_("Populate done at %s"%(time.time()-time_))
+                 logger.info("Populate done at %s"%(time.time()-time_))

                  """
                  randomize = i%100 < 40
                  if    i%100 < 40 :
@@ -3862,9 +3878,8 @@
                  else:
                        population.test(population.test_spiece_centroid)

-                 print_("Test done at %s"%(time.time()-time_))
+                 logger.info("Test done at %s"%(time.time()-time_))
                  draw_new_champ = False
```

```
-                print_()


                 if population.population[0][0]!= last_champ :
@@ -3873,10 +3888,9 @@
                      last_champ = population.population[0][0]*1


-                print_("Cicle %s done in %s"%(i,time.time()-time_))
+                logger.info("Cicle %s done in %s"%(i,time.time()-time_))
                 time_ = time.time()
-                print_("%s incests been found"%population.inc)
-                print_()
+                logger.info("%s incests been found"%population.inc)

                 if i == 0  or i == population_count-1 or draw_new_champ :
                      colors = ["blue"]
@@ -3927,9 +3941,10 @@

      def __init__(self):
           inkex.Effect.__init__(self)
-          self.OptionParser.add_option("-d", "--directory",
      action="store", type="string",         dest="directory",
default="/home/",                            help="Directory for gcode file")
+          self.OptionParser.add_option("-d", "--directory",
      action="store", type="string",         dest="directory", default="/home",
                            help="Directory for gcode file")
           self.OptionParser.add_option("-f", "--filename",
      action="store", type="string",         dest="file", default="-1.0",
                      help="File name")
           self.OptionParser.add_option("",   "--add-numeric-suffix-to-
filename", action="store", type="inkbool",
      dest="add_numeric_suffix_to_filename", default=True,help="Add numeric
suffix to filename")
+          self.OptionParser.add_option("", "--load-in-emc", action="store",
type="inkbool", dest="load_into_emc", default=False, help="Load output into
AXIS on completion.")
           self.OptionParser.add_option("",   "--Zscale",
           action="store", type="float",          dest="Zscale",
default="1.0",                               help="Scale factor Z")

           self.OptionParser.add_option("",   "--Zoffset",
           action="store", type="float",          dest="Zoffset",
default="0.0",                               help="Offset along Z")
           self.OptionParser.add_option("-s", "--Zsafe",
      action="store", type="float",          dest="Zsafe", default="0.5",
                      help="Z above all obstacles")
@@ -3986,7 +4001,7 @@
           self.OptionParser.add_option("",   "--lathe-rectangular-cutter-
width",action="store", type="float",   dest="lathe_rectangular_cutter_width",
default="4",          help="Rectangular cutter width")
```

```
            self.OptionParser.add_option("",   "--create-log",
      action="store", type="inkbool",   dest="log_create_log", default=False,
                  help="Create log files")
-           self.OptionParser.add_option("",   "--log-filename",
      action="store", type="string",         dest="log_filename", default='',
                        help="Create log files")
+           self.OptionParser.add_option("",   "--log-filename",
      action="store", type="string",         dest="log_filename",
default='/var/log/gcodetools.log',                            help="Create log
files")

            self.OptionParser.add_option("",   "--orientation-points-count",
      action="store", type="string",         dest="orientation_points_count",
default="2",                help="Orientation points count")
            self.OptionParser.add_option("",   "--tools-library-type",
      action="store", type="string",         dest="tools_library_type",
default='cylinder cutter',  help="Create tools definition")
@@ -4099,7 +4114,7 @@
                              sp2 = [  [subpath[i  ][j][0], subpath[i  ][j][1]]
for j in range(3)]
                              c += biarc(sp1,sp2,0,0) if w==None else
biarc(sp1,sp2,-f(w[k][i-1]),-f(w[k][i]))
 #                            l1 = biarc(sp1,sp2,0,0) if w==None else
biarc(sp1,sp2,-f(w[k][i-1]),-f(w[k][i]))
-#                            print_((-f(w[k][i-1]),-f(w[k][i]), [i1[5] for i1
in l1]) )
+#                            logger.info(str((-f(w[k][i-1]),-f(w[k][i]), [i1[5]
for i1 in l1]) ))
                        c += [ [ [subpath[-1][1][0],subpath[-1][1][1]]
,'end',0,0] ]
                  return c

@@ -4211,7 +4226,7 @@
                        self.options.directory += "\\"
                  else :
                        self.options.directory += "/"
-           print_("Checking directory: '%s'"%self.options.directory)
+           logger.info("Checking directory: '%s'"%self.options.directory)
            if (os.path.isdir(self.options.directory)):
                  if (os.path.isfile(self.options.directory+'header')):
                        f = open(self.options.directory+'header', 'r')
@@ -4227,7 +4242,7 @@
                        self.footer = defaults['footer']
                  self.header += self.options.unit + "\n"
            else:
-                 self.error(_("Directory does not exist! Please specify
existing directory at Preferences tab!"),"error")
+                 logger.error("Directory does not exist! Please specify
existing directory at Preferences tab!")
                  return False

            if self.options.add_numeric_suffix_to_filename :
```

```
@@ -4257,7 +4272,7 @@
                 f = open(self.options.directory+self.options.file, "w")
                 f.close()
           except:
-                self.error(_("Can not write to specified
file!\n%s"%(self.options.directory+self.options.file)),"error")
+                logger.error("Can not write to specified file!\n%s",
self.options.directory + self.options.file)
                 return False
           return True

@@ -4298,8 +4313,8 @@
                 self.last_used_tool == None
           except :
                 self.last_used_tool = None
-          print_("working on curve")
-          print_(curve)
+          logger.info("working on curve")
+          logger.debug(str(curve))

           if tool != self.last_used_tool :
                 g += ( "(Change tool to %s)\n" % re.sub("\"'\(\)\\\\","
",tool["name"]) ) + tool["tool change gcode"] + "\n"
@@ -4370,7 +4385,7 @@
                       t = g.get('transform')
                       t = simpletransform.parseTransform(t)
                       trans = simpletransform.composeTransform(t,trans) if
trans != [] else t
-                      print_(trans)
+                      logger.debug(str(trans))
                 g=g.getparent()
           return trans

@@ -4403,21 +4418,21 @@
                       if self.layers[i] in self.orientation_points :
                             break
                 if self.layers[i] not in self.orientation_points :
-                      self.error(_("Orientation points for '%s' layer have not
been found! Please add orientation points using Orientation tab!") %
layer.get(inkex.addNS('label','inkscape')),"no_orientation_points")
+                      logger.error("Orientation points for '%s' layer have not
been found! Please add orientation points using Orientation tab!",
layer.get(inkex.addNS('label','inkscape')))
                 elif self.layers[i] in self.transform_matrix :
                       self.transform_matrix[layer] =
self.transform_matrix[self.layers[i]]
                       self.Zcoordinates[layer] =
self.Zcoordinates[self.layers[i]]
                 else :
                       orientation_layer = self.layers[i]
                       if len(self.orientation_points[orientation_layer])>1 :
```

```
-                           self.error(_("There are more than one orientation
point groups in '%s' layer") %
orientation_layer.get(inkex.addNS('label','inkscape')),"more_than_one_orientati
on_point_groups")
+                           logger.error("There are more than one orientation
point groups in '%s' layer",
orientation_layer.get(inkex.addNS('label','inkscape')))
                       points = self.orientation_points[orientation_layer][0]
                       if len(points)==2:
                             points += [ [ [(points[1][0][1]-
points[0][0][1])+points[0][0][0], -(points[1][0][0]-
points[0][0][0])+points[0][0][1]], [-(points[1][1][1]-
points[0][1][1])+points[0][1][0], points[1][1][0]-
points[0][1][0]+points[0][1][1]] ] ]
                       if len(points)==3:
-                            print_("Layer '%s' Orientation points: " %
orientation_layer.get(inkex.addNS('label','inkscape')))
+                            logger.debug("Layer '%s' Orientation points: " %
orientation_layer.get(inkex.addNS('label','inkscape')))
                             for point in points:
-                                  print_(point)
+                                  logger.debug(str(point))
                             #     Zcoordinates definition taken from
Orientatnion point 1 and 2
                             self.Zcoordinates[layer] =
[max(points[0][1][2],points[1][1][2]), min(points[0][1][2],points[1][1][2])]
                             matrix = numpy.array([
@@ -4441,19 +4456,18 @@
                                  self.transform_matrix[layer] = [[m[j*3+i][0]
for i in range(3)] for j in range(3)]

                             else :
-                                  self.error(_("Orientation points are wrong!
(if there are two orientation points they should not be the same. If there are
three orientation points they should not be in a straight
line.)"),"wrong_orientation_points")
+                                  logger.error("Orientation points are wrong!
(if there are two orientation points they should not be the same. If there are
three orientation points they should not be in a straight line.)")
                       else :
-                            self.error(_("Orientation points are wrong! (if
there are two orientation points they should not be the same. If there are
three orientation points they should not be in a straight
line.)"),"wrong_orientation_points")
-
+                            logger.error("Orientation points are wrong! (if
there are two orientation points they should not be the same. If there are
three orientation points they should not be in a straight line.)")
                 self.transform_matrix_reverse[layer] =
numpy.linalg.inv(self.transform_matrix[layer]).tolist()
-                print_("\n Layer '%s' transformation matrixes:" %
layer.get(inkex.addNS('label','inkscape')) )
```

```
-                print_(self.transform_matrix)
-                print_(self.transform_matrix_reverse)
+                logger.debug("Layer '%s' transformation matrixes:" %
layer.get(inkex.addNS('label','inkscape')) )
+                logger.debug(self.transform_matrix)
+                logger.debug(self.transform_matrix_reverse)

                 ###self.Zauto_scale[layer]  = math.sqrt(
(self.transform_matrix[layer][0][0]**2 +
self.transform_matrix[layer][1][1]**2)/2 )
                 ### Zautoscale is absolete
                 self.Zauto_scale[layer] = 1
-                print_("Z automatic scale = %s (computed according orientation
points)" % self.Zauto_scale[layer])
+                logger.info("Z automatic scale = %s (computed according
orientation points)" % self.Zauto_scale[layer])

           x,y = source_point[0], source_point[1]
           if not reverse :
@@ -4470,54 +4484,6 @@
                       for k in xrange(len(csp[i][j])):
                             csp[i][j][k] = self.transform(csp[i][j][k],layer,
reverse)
           return csp
-
-
-
################################################################################
#
-###       Errors handling function, notes are just printed into Logfile,
-###       warnings are printed into log file and warning message is displayed
but
-###       extension continues working, errors causes log and execution is
halted
-###       Notes, warnings adn errors could be assigned to space or comma or
dot
-###       sepparated strings (case is ignoreg).
-
################################################################################
#
-     def error(self, s, type_= "Warning"):
-          notes = "Note "
-          warnings = """
-                                 Warning tools_warning
-                                 orientation_warning
-                                 bad_orientation_points_in_some_layers
-                                 more_than_one_orientation_point_groups
-                                 more_than_one_tool
-                                 orientation_have_not_been_defined
-                                 tool_have_not_been_defined
-                                 selection_does_not_contain_paths
```

```
-
      selection_does_not_contain_paths_will_take_all
-                                 selection_is_empty_will_comupe_drawing
-                                 selection_contains_objects_that_are_not_paths
-                                 Continue
-                                 """
-          errors = """
-                                 Error
-                                 wrong_orientation_points
-                                 area_tools_diameter_error
-                                 no_tool_error
-                                 active_layer_already_has_tool
-                                 active_layer_already_has_orientation_points
-                            """
-          s = str(s)
-          if type_.lower() in re.split("[\s\n,\.]+", errors.lower()) :
-               print_(s)
-               inkex.errormsg(s+"\n")
-               sys.exit()
-          elif type_.lower() in re.split("[\s\n,\.]+", warnings.lower()) :
-               print_(s)
-               inkex.errormsg(s+"\n")
-          elif type_.lower() in re.split("[\s\n,\.]+", notes.lower()) :
-               print_(s)
-          else :
-               print_(s)
-               inkex.errormsg(s)
-               sys.exit()
-


##############################################################################
#
 ###       Set markers
@@ -4611,22 +4577,24 @@
                            points = self.get_orientation_points(i)
                            if points != None :
                                 self.orientation_points[layer] =
self.orientation_points[layer]+[points[:]] if layer in self.orientation_points
else [points[:]]
-                                print_("Found orientation points in '%s'
layer: %s" % (layer.get(inkex.addNS('label','inkscape')), points))
+                                logger.info("Found orientation points in
'%s'", layer.get(inkex.addNS('label','inkscape')))
+                                logger.debug("Orientation points: %s",
points)
                            else :
-                                self.error(_("Warning! Found bad orientation
points in '%s' layer. Resulting Gcode could be corrupt!") %
layer.get(inkex.addNS('label','inkscape')),
"bad_orientation_points_in_some_layers")
```

```
+                                 logger.warning("Warning! Found bad
orientation points in '%s' layer. Resulting Gcode could be corrupt!",
layer.get(inkex.addNS('label','inkscape')))

                       #Need to recognise old files ver 1.6.04 and earlier
                       elif i.get("gcodetools") == "Gcodetools tool definition"
or i.get("gcodetools") == "Gcodetools tool defenition"  :
                            tool = self.get_tool(i)
                            self.tools[layer] = self.tools[layer] +
[tool.copy()] if layer in self.tools else [tool.copy()]
-                           print_("Found tool in '%s' layer: %s" %
(layer.get(inkex.addNS('label','inkscape')), tool))
+                           logger.info("Found tool in '%s'
layer",layer.get(inkex.addNS('label','inkscape')))
+                           logger.debug("Tool: %s", tool)

                       elif i.get("gcodetools") == "Gcodetools graffiti
reference point" :
                            point = self.get_graffiti_reference_points(i)
                            if point != [] :
                                 self.graffiti_reference_points[layer] =
self.graffiti_reference_points[layer]+[point[:]] if layer in
self.graffiti_reference_points else [point]
                            else :
-                                self.error(_("Warning! Found bad graffiti
reference point in '%s' layer. Resulting Gcode could be corrupt!") %
layer.get(inkex.addNS('label','inkscape')),
"bad_orientation_points_in_some_layers")
+                                logger.warning("Warning! Found bad graffiti
reference point in '%s' layer. Resulting Gcode could be corrupt!",
layer.get(inkex.addNS('label','inkscape')))

                       elif i.tag == inkex.addNS('path','svg'):
                            if "gcodetools"  not in i.keys() :
@@ -4647,17 +4615,17 @@

                       elif i.get("id") in self.selected :
 # xgettext:no-pango-format
-                           self.error(_("This extension works with Paths and
Dynamic Offsets and groups of them only! All other objects will be
ignored!\nSolution 1: press Path->Object to path or Shift+Ctrl+C.\nSolution 2:
Path->Dynamic offset or Ctrl+J.\nSolution 3: export all contours to PostScript
level 2 (File->Save As->.ps) and File->Import this
file."),"selection_contains_objects_that_are_not_paths")
+                           logger.warning("This extension works with Paths
and Dynamic Offsets and groups of them only! All other objects will be
ignored!\nSolution 1: press Path->Object to path or Shift+Ctrl+C.\nSolution 2:
Path->Dynamic offset or Ctrl+J.\nSolution 3: export all contours to PostScript
level 2 (File->Save As->.ps) and File->Import this file.")


            recursive_search(self.document.getroot(),self.document.getroot())
```

```
            if len(self.layers) == 1 :
-                 self.error(_("Document has no layers! Add at least one layer
using layers panel (Ctrl+Shift+L)"),"Error")
+                 logger.error("Document has no layers! Add at least one layer
using layers panel (Ctrl+Shift+L)")
            root = self.document.getroot()

            if  root in self.selected_paths or root in self.paths :
-                 self.error(_("Warning! There are some paths in the root of the
document, but not in any layer! Using bottom-most layer for them."),
"tools_warning" )
+                 logger.warning("Warning! There are some paths in the root of
the document, but not in any layer! Using bottom-most layer for them.")

            if  root in self.selected_paths :
                  if self.layers[-1] in self.selected_paths :
@@ -4728,33 +4696,34 @@
                                    value = get_text(j)
                                    if value == "(None)": value = ""
                        if value == None or key == None: continue
-                       #print_("Found tool parameter '%s':'%s'" % (key,value))
+                       #logger.info("Found tool parameter '%s':'%s'" %
(key,value))
                        if key in self.default_tool.keys() :
                               try :
                                    tool[key] =
type(self.default_tool[key])(value)
                               except :
                                    tool[key] = self.default_tool[key]
-                                   self.error(_("Warning! Tool's and default
tool's parameter's (%s) types are not the same ( type('%s') != type('%s') ).")
% (key, value, self.default_tool[key]), "tools_warning")
+                                   logger.warning("Warning! Tool's and default
tool's parameter's (%s) types are not the same ( type('%s') != type('%s') ).",
key, value, self.default_tool[key])
                        else :
                              tool[key] = value
-                             self.error(_("Warning! Tool has parameter that
default tool has not ( '%s': '%s' ).") % (key, value), "tools_warning" )
+                             logger.warning("Warning! Tool has parameter that
default tool has not ( '%s': '%s' ).", key,  value)
            return tool


      def set_tool(self,layer):
-#
      print_(("index(layer)=",self.layers.index(layer),"set_tool():layer=",laye
r,"self.tools=",self.tools))
+#         logger.info("index(layer)=%s set_tool():layer= %s self.tools=%s",
self.layers.index(layer), layer, self.tools)
 #         for l in self.layers:
```

```
-#                print_(("l=",l))
+#                logger.info("l=%s",l)
            for i in range(self.layers.index(layer),-1,-1):
-#                print_(("processing layer",i))
+#                logger.info("processing layer %s",i)
                  if self.layers[i] in self.tools :
                        break
            if self.layers[i] in self.tools :
                  if self.layers[i] != layer : self.tools[layer] =
self.tools[self.layers[i]]
-                 if len(self.tools[layer])>1 : self.error(_("Layer '%s'
contains more than one tool!") %
self.layers[i].get(inkex.addNS('label','inkscape')), "more_than_one_tool")
+                 if len(self.tools[layer])>1:
+                       logger.warning("Layer '%s' contains more than one
tool!", self.layers[i].get(inkex.addNS('label','inkscape')))
                  return self.tools[layer]
            else :
-                 self.error(_("Can not find tool for '%s' layer! Please add one
with Tools library tab!") % layer.get(inkex.addNS('label','inkscape')),
"no_tool_error")
+                 logger.error("Can not find tool for '%s' layer! Please add one
with Tools library tab!", layer.get(inkex.addNS('label','inkscape')))



################################################################################
#
@@ -4814,7 +4783,7 @@

            def sort_dxfpoints(points):
                  points=remove_duplicates(points)
-#                 print_(get_boundaries(get_boundaries(points)[2])[1])
+#                 logger.info(str(get_boundaries(get_boundaries(points)[2])[1]))
                  ways=[
                                          # l=0, d=1, r=2, u=3
                   [3,0], # ul
@@ -4826,17 +4795,17 @@
                   [2,3], # ru
                   [2,1], # rd
                  ]
-#                 print_(("points=",points))
+#                 logger.info("points=%s",points)
                  minimal_way=[]
                  minimal_len=None
                  minimal_way_type=None
                  for w in ways:
                        tpoints=points[:]
                        cw=[]
-#                       print_(("tpoints=",tpoints))
+#                       logger.info("tpoints=%s",tpoints)
                        for j in xrange(0,len(points)):
```

```
      p=get_boundaries(get_boundaries(tpoints)[w[0]])[w[1]]
-#                          print_(p)
+#                          logger.info(str(p))
                           tpoints.remove(p[0])
                           cw+=p
                      curlen = get_way_len(cw)
@@ -4852,7 +4821,7 @@
                 lines = [ [key]+lines[key] for key in range(len(lines))]

                 keys = [0]
                 end_point = lines[0][3:]
-                print_("!!!",lines,"\n",end_point)
+                logger.debug("!!! %s : %s",lines,end_point)
                 del lines[0]
                 while len(lines)>0:
                      dist = [ [point_to_point_d2(end_point,lines[i][1:3]),i]
for i in range(len(lines))]
@@ -4872,7 +4841,7 @@
                 gcode=""
                 for point in points:
                      gcode +="(drilling dxfpoint)\nG00 Z%f\nG00 X%f Y%f\nG01
Z%f F%f\nG04 P%f\nG00 Z%f\n" %
(self.options.Zsafe,point[0],point[1],self.Zcoordinates[layer][1],self.tools[la
yer][0]["penetration feed"],0.2,self.options.Zsafe)
-#                print_(("got dxfpoints array=",points))
+#                logger.info("got dxfpoints array= %s",points)
                 return gcode

           def get_path_properties(node, recursive=True,
tags={inkex.addNS('desc','svg'):"Description",inkex.addNS('title','svg'):"Title
"} ) :
@@ -4889,19 +4858,19 @@

           if self.selected_paths == {} and self.options.auto_select_paths:
                 paths=self.paths
-                self.error(_("No paths are selected! Trying to work on all
available paths."),"warning")
+                logger.warning("No paths are selected! Trying to work on all
available paths.")
           else :
                 paths = self.selected_paths
           self.check_dir()
           gcode = ""

           biarc_group = inkex.etree.SubElement( self.selected_paths.keys()[0]
if len(self.selected_paths.keys())>0 else self.layers[0],
inkex.addNS('g','svg') )
-          print_(("self.layers=",self.layers))
-          print_(("paths=",paths))
+          logger.debug("self.layers = %s",self.layers)
+          logger.debug("paths = %s", paths)
```

```
            colors = {}
            for layer in self.layers :
                  if layer in paths :
-                       print_(("layer",layer))
+                       logger.debug("layer %s",layer)
                        # transform simple path to get all var about orientation
                        self.transform_csp([ [ [[0,0],[0,0],[0,0]],
[[0,0],[0,0],[0,0]] ] ], layer)

@@ -4912,11 +4881,11 @@
                        try :
                              depth_func = eval('lambda c,d,s: ' +
self.options.path_to_gcode_depth_function.strip('"'))
                        except:
-                             self.error("Bad depth function! Enter correct
function at Path to Gcode tab!")
+                             logger.error("Bad depth function! Enter correct
function at Path to Gcode tab!")

                        for path in paths[layer] :
                              if "d" not in path.keys() :
-                                   self.error(_("Warning: One or more paths do
not have 'd' parameter, try to Ungroup (Ctrl+Shift+G) and Object to Path
(Ctrl+Shift+C)!"),"selection_contains_objects_that_are_not_paths")
+                                   logger.warning("Warning: One or more paths do
not have 'd' parameter, try to Ungroup (Ctrl+Shift+G) and Object to Path
(Ctrl+Shift+C)!")
                                    continue
                              csp = cubicsuperpath.parsePath(path.get("d"))
                              csp = self.apply_transforms(path, csp)
@@ -4944,7 +4913,7 @@
                                    tmp_curve=self.transform_csp(csp, layer)
                                    x=tmp_curve[0][0][0][0]
                                    y=tmp_curve[0][0][0][1]
-                                   print_("got dxfpoint (scaled) at (%f,%f)" %
(x,y))
+                                   logger.info("got dxfpoint (scaled) at
(%f,%f)" % (x,y))
                                    dxfpoints += [[x,y]]
                              else:

@@ -4957,7 +4926,7 @@
                                                            ]
                                                      ]
 #                      for c in curves :
-#                           print_(c)
+#                           logger.info(str(c))
                        dxfpoints=sort_dxfpoints(dxfpoints)
                        gcode+=print_dxfpoints(dxfpoints)

@@ -5030,18 +4999,18 @@
```

```
##############################################################################
#
      def dxfpoints(self):
            if self.selected_paths == {}:
-                 self.error(_("Noting is selected. Please select something to
convert to drill point (dxfpoint) or clear point sign."),"warning")
+                 logger.warning("Noting is selected. Please select something to
convert to drill point (dxfpoint) or clear point sign.")
            for layer in self.layers :
                  if layer in self.selected_paths :
                        for path in self.selected_paths[layer]:
-#                            print_(("processing path",path.get('d')))
+#                            logger.info("processing path %s",path.get('d'))
                             if self.options.dxfpoints_action == 'replace':
-#                                  print_("trying to set as dxfpoint")
+#                                  logger.info("trying to set as dxfpoint")

                                   path.set("dxfpoint","1")
                                   r = re.match("^\s*.\s*(\S+)",path.get("d"))
                                   if r!=None:
-                                        print_(("got path=",r.group(1)))
+                                        logger.info("got path=%s",r.group(1))
                                         path.set("d","m %s 2.9375,-
6.343750000001 0.8125,1.90625 6.843748640396,-6.84374864039 0,0 0.6875,0.6875 -
6.84375,6.84375 1.90625,0.812500000001 z" % r.group(1))
                                         path.set("style",styles["dxf_points"])

@@ -5051,7 +5020,7 @@
                             if self.options.dxfpoints_action == 'clear' and
path.get("dxfpoint") == "1":
                                   path.set("dxfpoint","0")
 #                                 for id, node in self.selected.iteritems():
-#                                        print_((id,node,node.attrib))
+#                                        logger.info("%s %s %s",
id,node,node.attrib)


##############################################################################
#
@@ -5062,7 +5031,7 @@
      def area_artefacts(self) :
                  if self.selected_paths == {} and
self.options.auto_select_paths:
                        paths=self.paths
-                       self.error(_("No paths are selected! Trying to work on
all available paths."),"warning")
+                       logger.warning("No paths are selected! Trying to work on
all available paths.")
                  else :
                        paths = self.selected_paths
```

```
                 for layer in paths :
@@ -5071,7 +5040,7 @@
                            parent = path.getparent()
                            style = path.get("style") if "style" in
path.keys() else ""
                            if "d" not in path.keys() :
-                                 self.error(_("Warning: One or more paths do
not have 'd' parameter, try to Ungroup (Ctrl+Shift+G) and Object to Path
(Ctrl+Shift+C)!"),"selection_contains_objects_that_are_not_paths")
+                                 logger.warning("Warning: One or more paths do
not have 'd' parameter, try to Ungroup (Ctrl+Shift+G) and Object to Path
(Ctrl+Shift+C)!")
                                  continue
                            csp = cubicsuperpath.parsePath(path.get("d"))
                            remove = []
@@ -5094,7 +5063,7 @@
                                            remove.append(i)
                                       elif
self.options.area_find_artefacts_action == "delete" :
                                            remove.append(i)
-                                           print_("Deleted artefact %s" %
subpath )
+                                           logger.info("Deleted artefact %s"
% subpath )
                            remove.reverse()
                            for i in remove :
                                  del csp[i]
@@ -5113,29 +5082,28 @@

##############################################################################
#
      def area(self) :
            if len(self.selected_paths)<=0:
-                 self.error(_("This extension requires at least one selected
path."),"warning")
+                 logger.warning("This extension requires at least one selected
path.")
                  return
            for layer in self.layers :
                  if layer in self.selected_paths :
                        self.set_tool(layer)
                        if self.tools[layer][0]['diameter']<=0 :
-                             self.error(_("Tool diameter must be > 0 but tool's
diameter on '%s' layer is not!") %
layer.get(inkex.addNS('label','inkscape')),"area_tools_diameter_error")
+                             logger.error("Tool diameter must be > 0 but tool's
diameter on '%s' layer is not!",layer.get(inkex.addNS('label','inkscape')))

                        for path in self.selected_paths[layer]:
-                             print_(("doing path",  path.get("style"),
path.get("d")))
```

```
+                            logger.info("doing path %s %s",
      path.get("style"), path.get("d"))

                             area_group = inkex.etree.SubElement(
path.getparent(), inkex.addNS('g','svg') )

                             d = path.get('d')
-                            print_(d)
+                            logger.info(str(d))
                             if d==None:
-                                  print_("omitting non-path")
-                                  self.error(_("Warning: omitting non-
path"),"selection_contains_objects_that_are_not_paths")
+                                  logger.warning("Warning: omitting non-path")
                                   continue
                             csp = cubicsuperpath.parsePath(d)

                             if
path.get(inkex.addNS('type','sodipodi'))!="inkscape:offset":
-                                  print_("Path %s is not an offset. Preparation
started." % path.get("id"))
+                                  logger.info("Path %s is not an offset.
Preparation started." % path.get("id"))
                                   # Path is not offset. Preparation will be
needed.
                                   # Finding top most point in path (min y
value)

@@ -5167,22 +5135,22 @@


                                   d = cubicsuperpath.formatPath(csp)
-                                  print_(("original  d=",d))
+                                  logger.info("original  d=%s",d)
                                   d = re.sub(r'(?i)(m[^mz]+)',r'\1 Z ',d)
                                   d = re.sub(r'(?i)\s*z\s*z\s*',r' Z ',d)
                                   d = re.sub(r'(?i)\s*([A-Za-z])\s*',r' \1 ',d)
-                                  print_(("formatted d=",d))
+                                  logger.info("formatted d=%s",d)
                             # scale = sqrt(Xscale**2 + Yscale**2) / sqrt(1**2
+ 1**2)
                             p0 = self.transform([0,0],layer)
                             p1 = self.transform([0,1],layer)
                             scale = (P(p0)-P(p1)).mag()
                             if scale == 0 : scale = 1.
                             else : scale = 1./scale
-                            print_(scale)
+                            logger.info(str(scale))
                             tool_d = self.tools[layer][0]['diameter']*scale
                             r = self.options.area_inkscape_radius * scale
                             sign=1 if r>0 else -1
-                            print_("Tool diameter = %s, r = %s" % (tool_d, r))
```

```
+                           logger.info("Tool diameter = %s, r = %s" %
(tool_d, r))

                            # avoiding infinite loops
                            if self.options.area_tool_overlap>0.9 :
self.options.area_tool_overlap = .9
@@ -5199,7 +5167,7 @@

inkex.addNS('original','inkscape'):    d,
                                                                  'style':
styles["biarc_style_i"]['area']
                                                            })
-                                 print_(("adding
curve",area_group,d,styles["biarc_style_i"]['area']))
+                                 logger.info("adding curve %s %s
%s",area_group,d,styles["biarc_style_i"]['area'])
                                  if radius == -r : break


@@ -5318,8 +5286,7 @@
                       for path in self.selected_paths[layer]:
                            d = path.get('d')
                            if d==None:
-                                 print_("omitting non-path")
-                                 self.error(_("Warning: omitting non-
path"),"selection_contains_objects_that_are_not_paths")
+                                 logger.warning("Warning: omitting non-path")
                                  continue
                            csp = cubicsuperpath.parsePath(d)
                            csp = self.apply_transforms(path, csp)
@@ -5359,22 +5326,21 @@
            # convert degrees into rad
            self.options.area_fill_angle = self.options.area_fill_angle *
math.pi / 180
            if len(self.selected_paths)<=0:
-                 self.error(_("This extension requires at least one selected
path."),"warning")
+                 logger.warning("This extension requires at least one selected
path.")
                  return
            for layer in self.layers :
                  if layer in self.selected_paths :
                        self.set_tool(layer)
                        if self.tools[layer][0]['diameter']<=0 :
-                             self.error(_("Tool diameter must be > 0 but tool's
diameter on '%s' layer is not!") %
layer.get(inkex.addNS('label','inkscape')),"area_tools_diameter_error")
+                             logger.error("Tool diameter must be > 0 but tool's
diameter on '%s' layer is not!", layer.get(inkex.addNS('label','inkscape')))
                        tool = self.tools[layer][0]
                        for path in self.selected_paths[layer]:
                              lines = []
```

```
-                           print_(("doing path",  path.get("style"),
path.get("d")))
+                           logger.info("doing path %s %s", path.get("style"),
path.get("d"))
                            area_group = inkex.etree.SubElement(
path.getparent(), inkex.addNS('g','svg') )
                            d = path.get('d')
                            if d==None:
-                                 print_("omitting non-path")
-                                 self.error(_("Warning: omitting non-
path"),"selection_contains_objects_that_are_not_paths")
+                                 logger.warning("Warning: omitting non-path")
                                  continue
                            csp = cubicsuperpath.parsePath(d)
                            csp = self.apply_transforms(path, csp)
@@ -5398,7 +5364,7 @@
                            # Zig-zag
                            r = tool['diameter']*(1-
self.options.area_tool_overlap)
                            if r<=0 :
-                                 self.error('Tools diameter must be greater
than 0!', 'error')
+                                 logger.error('Tools diameter must be greater
than 0!')
                                  return

                            lines += [ [] ]
@@ -5488,7 +5454,7 @@
                                       splitted_line += [ [ [ i[1], i[2]] ] ]
                                 splitted_line[-1] += [  l2  ]
                                 i = 0
-                           print_(splitted_line)
+                           logger.info(str(splitted_line))
                            while i < len(splitted_line) :
                                 # check if the middle point of the first
lines segment is inside the path.
                                 # and remove the subline if not.
@@ -5666,14 +5632,14 @@
                            return math.sqrt(x2**2+y2**2)
                      #if x2,y2 not in front of the normal...
                      if x2*nx+y2*ny <=0 : return max_dist
-                     #print_("Straight",x1,y1,nx,ny,x2,y2)
+                     #logger.info("Straight %f %f %f %f %f
%f",x1,y1,nx,ny,x2,y2)
                      return (x2**2+y2**2)/(2*(x2*nx+y2*ny) )
                 #It is a corner bisector, so..
                 discriminator = (x2*nx+y2*ny)**2 - denom*(x2**2+y2**2)
                 if discriminator < 0 :
                      return max_dist #this part irrelevant
                 r=(x2*nx+y2*ny -math.sqrt(discriminator))/denom
-                #print_("Corner",x1,y1,nx,ny,x1+x2,y1+y2,discriminator,r)
```

```
+                #logger.info("Corner" + "
%f"*8,x1,y1,nx,ny,x1+x2,y1+y2,discriminator,r)
                 return min(r, max_dist)
                 #end of get_radius_to_point

@@ -5742,7 +5708,7 @@
                                  if jj==j : #except this one
                                        if abs(ii-i)<3 or abs(ii-
i)>len(nlLT[j])-3 : continue

     t1=get_radius_to_point((x1,y1),(nx,ny),nlLT[jj][ii-1][0] )
-                                 #print_("Try pt
i,ii,t1,x1,y1",i,ii,t1,x1,y1)
+#                                logger.info("Try pt i,ii,t1,x1,y1=" + "
%f"*5, i,ii,t1,x1,y1)
                            else: #doing a line
                                  if jj==j : #except this one
                                        if abs(ii-i)<2 or abs(ii-
i)==len(nlLT[j])-1 : continue
@@ -5764,7 +5730,7 @@
                                  #see if line in range
                                  if n1[2]==False and (x23max<xmin or
x23min>xmax or y23max<ymin or y23min>ymax) : continue
                                  t1=get_radius_to_line((x1,y1),(nx,ny),
(nx2,ny2),(x2,y2),(nx23,ny23), (x3,y3),(nx3,ny3))
-                                 #print_("Try line
i,ii,t1,x1,y1",i,ii,t1,x1,y1)
+                                 #logger.info("Try line i,ii,t1,x1,y1 = " + "
%f"*5,i,ii,t1,x1,y1)
                            if 0<=t1<r :
                                  r = t1
                                  iimin = ii
@@ -5829,7 +5795,7 @@
                                        if xydist<engraving_tolerance : #so far
so good
                                              wdist=w2+(w-w2)*length1/length2 -
w1
                                              if abs(wdist)<engraving_tolerance
:
-                                                   #print_("pop",j,i,xy1)
+                                                   #logger.info("pop" + "
%f"*3,j,i,xy1)
                                                    cspm.pop()
                                                    wl.pop()
                 cspm+=[ [ [x,y],[x,y],[x,y],i,j,ii,jj ] ]
@@ -5874,7 +5840,7 @@
           cspe =[]
           we = []
           if len(self.selected_paths)<=0:
-                self.error(_("Please select at least one path to engrave and
run again."),"warning")
```

```
+                 logger.warning("Please select at least one path to engrave and
run again.")
                  return
            if not self.check_dir() : return
            #Find what units the user uses
@@ -5882,8 +5848,8 @@
            if self.options.unit == "G20 (All units in inches)" :
                  unit=" inches"
            elif self.options.unit != "G21 (All units in mm)" :
-                 self.error(_("Unknown unit selected. mm assumed"),"warning")
-           print_("engraving_max_dist mm/inch",
self.options.engraving_max_dist )
+                 logger.warning("Unknown unit selected. mm assumed")
+           logger.info("engraving_max_dist mm/inch %s",
self.options.engraving_max_dist )

            #LT See if we can use this parameter for line and Bezier
subdivision:
            bitlen=20/self.options.engraving_newton_iterations
@@ -5895,7 +5861,7 @@
                        p2=self.orientation_points[layer][0][1]
                        ol=math.hypot(p1[0][0]-p2[0][0],p1[0][1]-p2[0][1])
                        oluu=math.hypot(p1[1][0]-p2[1][0],p1[1][1]-p2[1][1])
-                       print_("Orientation2 p1 p2 ol oluu",p1,p2,ol,oluu)
+                       logger.info("Orientation2 p1=%f p2=%f ol=%f
oluu=%f",p1,p2,ol,oluu)
                        orientation_scale = ol/oluu

                        self.set_tool(layer)
@@ -5903,15 +5869,15 @@
                        if re.search('w', shape) :
                              toolshape = eval('lambda w: ' + shape.strip('"'))
                        else:
-                             self.error(_("Tool '%s' has no shape. 45 degree
cone assumed!") % self.tools[layer][0]['name'],"Continue")
+                             logger.warning("Tool '%s' has no shape. 45 degree
cone assumed!", self.tools[layer][0]['name'])
                              toolshape = lambda w: w
                        #Get tool radius in pixels
                        toolr=self.tools[layer][0]['diameter'] *
orientation_scale/2
-                       print_("tool radius in pixels=", toolr)
+                       logger.info("tool radius in pixels %s", toolr)
                        #max dist from path to engrave in user's units
                        max_distuu = min(self.tools[layer][0]['diameter']/2,
self.options.engraving_max_dist)
                        max_dist=max_distuu*orientation_scale
-                       print_("max_dist pixels", max_dist )
+                       logger.info("max_dist pixels %d", max_dist )

                        engraving_group = inkex.etree.SubElement(
self.selected_paths[layer][0].getparent(), inkex.addNS('g','svg') )
```

```
                      if self.options.engraving_draw_calculation_paths and
(self.my3Dlayer  == None) :
@@ -5939,7 +5905,7 @@
                                                  i += 1
                                  for csp in cspi: #LT6a For each subpath...
                                        #Create copies in 3D layer
-                                       print_("csp is zz ",csp)
+                                       logger.info("csp is zz %s",csp)
                                        cspl=[]
                                        cspr=[]
                                        #create list containing lines and
points, starting with a point
@@ -5952,7 +5918,7 @@
                                        # if halfangle<0. reflex angle. normal
is bisector
                                        # corner normals are divided by
cos(halfangle)
                                        #so that they will engrave correctly
-                                       print_("csp is",csp)
+                                       logger.info("csp is %s",csp)
                                        nlLT.append ([])
                                        for i in range(0,len(csp)): #LT for
each point
                                              #n = []
@@ -5973,13 +5939,11 @@
                                              nx1,ny1 =
csp_normalized_normal(sp1,sp2,0)
                                              #I don't trust this function, so
test result
                                              if abs(1-math.hypot(nx1,ny1))>
0.00001 :
-
      print_("csp_normalised_normal error t=0",nx1,ny1,sp1,sp2)
-
      self.error(_("csp_normalised_normal error. See log."),"warning")
+
      logger.error("csp_normalised_normal error t=0, %f %f %f %f", nx1, ny1,
sp1, sp2)

                                              nx0, ny0 =
csp_normalized_normal(sp0,sp1,1)
                                              if abs(1-math.hypot(nx0,ny0))>
0.00001 :
-
      print_("csp_normalised_normal error t=1",nx0,ny0,sp1,sp2)
-
      self.error(_("csp_normalised_normal error. See log."),"warning")
+
      logger.info("csp_normalised_normal error t=1, %f %f %f
%f",nx0,ny0,sp1,sp2)

      bx,by,s=bisect((nx0,ny0),(nx1,ny1))
```

```
                                                #record x,y,normal,ifCorner,
sin(angle-turned/2)
                                                nlLT[-1] += [[ [x0,y0],[bx,by],
True, s]]
@@ -6016,10 +5980,10 @@

      bx,by,s=bisect((nx0,ny0),(nx1,ny1))
                                                                        nlLT[-1] +=
[[bLT[seg+1],[bx,by], True, 0.]]
                                          #LT for each segment - ends here.
-
      print_(("engraving_draw_calculation_paths=",self.options.engraving_draw_c
alculation_paths))
+
      logger.info("engraving_draw_calculation_paths=%s",self.options.engraving_
draw_calculation_paths)
                                          if
self.options.engraving_draw_calculation_paths:
                                                #Copy complete paths to 3D layer
-                                               #print_("cspl",cspl)
+                                               #logger.info("cspl %s",cspl)
                                                cspl+=[cspl[0]] #Close paths
                                                cspr+=[cspr[0]] #Close paths
                                                inkex.etree.SubElement(
      gcode_3Dleft , inkex.addNS('path','svg'),
@@ -6048,7 +6012,7 @@

                                    #LT6a build nlLT[j] for each subpath - ends
here
                                    #for nnn in nlLT :
-                                         #print_("nlLT",nnn) #LT debug stuff
+                                         #logger.info("nlLT %s",nnn) #LT debug
stuff
                                    # Calculate offset points
                                    reflex=False
                                    for j in xrange(len(nlLT)): #LT6b for each
subpath
@@ -6089,12 +6053,12 @@
                                                      bits=int((length-
bit0)/bitlen)
                                                      #split excess evenly at both
ends
                                                      bit0+=(length-bit0-
bitlen*bits)/2
-
      #print_("j,i,r,bit0,bits",j,i,w,bit0,bits)
+
      #logger.info("j,i,r,bit0,bits" + " %f"*5,j,i,w,bit0,bits)
                                                for b in xrange(bits) : #divide
line into bits
                                                      x1=x1a+ny*(b*bitlen+bit0)
                                                      y1=y1a-nx*(b*bitlen+bit0)
```

```
                                                        jjmin,iimin,w=get_biggest(
(x1,y1), (nx,ny))
-
      print_("i,j,jjmin,iimin,w",i,j,jjmin,iimin,w)
+
      logger.info("i=%f,j=%f,jmin=%f,iimin=%f,w=%f",i,j,jjmin,iimin,w)
                                                        #w = min(r, toolr)
                                                        wmax=max(wmax,w)
                                                        if reflex : #just after a
reflex corner
@@ -6123,9 +6087,9 @@
                                                              lastw = w #remember this w
                                                  #LT next i
                                                  cspm+=[cspm[0]]
-                                                 print_("cspm",cspm)
+                                                 logger.info("cspm %s",cspm)
                                                  wl+=[wl[0]]
-                                                 print_("wl",wl)
+                                                 logger.info("wl %s",wl)
                                                  #Note: Original csp_points was a list,
each element
                                                  #being 4 points, with the first being
the same as the
                                                  #last of the previous set.
@@ -6145,14 +6109,14 @@
                                                  wluu = [] #width list in user units:
mm/inches
                                                  for w in wl :
                                                        wluu+=[ w / orientation_scale ]
-                                                 print_("wl in pixels",wl)
-                                                 print_("wl in user units",wluu)
+                                                 logger.info("wl in pixels %s",wl)
+                                                 logger.info("wl in user units %s",wluu)
                                                  #LT previously, we was in pixels so
gave wrong depth
                                                  we   +=    [wluu]
                                            #LT6b For each subpath - ends here

                                      #LT5 if it is a path - ends here
-                                     #print_("cspe",cspe)
-                                     #print_("we",we)
+                                     #logger.info("cspe %s",cspe)
+                                     #logger.info("we %s",we)
                                #LT4 for each selected object in this layer - ends here

                                if cspe!=[]:
@@ -6167,8 +6131,8 @@
                            self.header+="(Rapid feeds use safe Z="+
str(self.options.Zsafe) + unit + ")\n"
                            self.header+="(Material surface at Z="+
str(self.options.Zsurface) + unit + ")\n"
                            self.export_gcode(gcode)
```

```
-          else :     self.error(_("No need to engrave sharp
angles."),"warning")
-
+          else :
+               logger.warning("No need to engrave sharp angles.")


##############################################################################
#
 ###
@@ -6186,8 +6150,8 @@
                 transform = simpletransform.formatTransform(transform)

           if self.options.orientation_points_count == "graffiti" :
-                print_(self.graffiti_reference_points)
-                print_("Inserting graffiti points")
+                logger.info(str(self.graffiti_reference_points))
+                logger.info("Inserting graffiti points")
                 if layer in self.graffiti_reference_points:
graffiti_reference_points_count =  len(self.graffiti_reference_points[layer])
                 else: graffiti_reference_points_count = 0
                 axis = ["X","Y","Z","A"][graffiti_reference_points_count%4]
@@ -6208,10 +6172,10 @@
                 draw_pointer(group = self.current_layer, x = self.view_center,
figure="arrow", pointer_type = "In-out reference point", text = "In-out point")

           else :
-                print_("Inserting orientation points")
+                logger.info("Inserting orientation points")

                 if layer in self.orientation_points:
-                      self.error(_("Active layer already has orientation
points! Remove them or select another
layer!"),"active_layer_already_has_orientation_points")
+                      logger.error("Active layer already has orientation
points! Remove them or select another layer!")

                 attr = {"gcodetools":"Gcodetools orientation group"}
                 if    transform != [] :
@@ -6221,18 +6185,18 @@
                 doc_height =
inkex.unittouu(self.document.getroot().get('height'))
                 if self.document.getroot().get('height') == "100%" :
                      doc_height = 1052.3622047
-                     print_("Overruding height from 100 percents to %s" %
doc_height)
+                     logger.info("Overruding height from 100 percents to %s"
% doc_height)
                 if self.options.unit == "G21 (All units in mm)" :
                      points =
[[0.,0.,self.options.Zsurface],[100.,0.,self.options.Zdepth],[0.,100.,0.]]
                      orientation_scale = 3.5433070660
```

```
-                       print_("orientation_scale < 0 ===> switching to mm
units=%0.10f"%orientation_scale )
+                       logger.info("orientation_scale < 0 ===> switching to mm
units=%0.10f"%orientation_scale )
                 elif self.options.unit == "G20 (All units in inches)" :
                       points =
[[0.,0.,self.options.Zsurface],[5.,0.,self.options.Zdepth],[0.,5.,0.]]
                       orientation_scale = 90
-                       print_("orientation_scale < 0 ===> switching to inches
units=%0.10f"%orientation_scale )
+                       logger.info("orientation_scale < 0 ===> switching to
inches units=%0.10f"%orientation_scale )
                 if self.options.orientation_points_count == "2" :
                       points = points[:2]
-                print_(("using orientation
scale",orientation_scale,"i=",points))
+                logger.info("using orientation scale %s
i=%s",orientation_scale,points)
                 for i in points :
                       si = [i[0]*orientation_scale, i[1]*orientation_scale]
                       g = inkex.etree.SubElement(orientation_group,
inkex.addNS('g','svg'), {'gcodetools': "Gcodetools orientation point (%s
points)" % self.options.orientation_points_count})
@@ -6256,7 +6220,7 @@
           if layer == None :
                 layer = self.current_layer if self.current_layer is not None
else self.document.getroot()
           if layer in self.tools:
-                self.error(_("Active layer already has a tool! Remove it or
select another layer!"),"active_layer_already_has_tool")
+                logger.error("Active layer already has a tool! Remove it or
select another layer!")

           if self.options.tools_library_type == "cylinder cutter" :
                 tool = {
@@ -6371,7 +6335,7 @@

################################################################################
#
      def check_tools_and_op(self):
           if len(self.selected)<=0 :
-                self.error(_("Selection is empty! Will compute whole
drawing."),"selection_is_empty_will_comupe_drawing")
+                logger.warning("Selection is empty! Will compute whole
drawing.")
                 paths = self.paths
           else :
                 paths = self.selected_paths
@@ -6425,7 +6389,7 @@
 ###       TODO Launch browser on help tab
```

```
##############################################################################
#
      def help(self):
-           self.error(_("""Tutorials, manuals and support can be found
at\nEnglish support forum:\nhttp://www.cnc-club.ru/gcodetools\nand Russian
support forum:\n http://www.cnc-club.ru/gcodetoolsru"""),"warning")
+           logger.info("""Tutorials, manuals and support can be found
at\nEnglish support forum:\nhttp://www.cnc-club.ru/gcodetools\nand Russian
support forum:\n http://www.cnc-club.ru/gcodetoolsru""")
            return


@@ -6470,10 +6434,10 @@
            x = re.sub("^\s*([XYZxyz])\s*$",r"\1",x)
            z = re.sub("^\s*([XYZxyz])\s*$",r"\1",z)
            if x not in ["X", "Y", "Z", "x", "y", "z"] or z not in ["X", "Y",
"Z", "x", "y", "z"] :
-                 self.error(_("Lathe X and Z axis remap should be 'X', 'Y' or
'Z'. Exiting..."),"warning")
+                 logger.warning("Lathe X and Z axis remap should be 'X', 'Y' or
'Z'. Exiting...")
                  return
            if x.lower() == z.lower() :
-                 self.error(_("Lathe X and Z axis remap should be the same.
Exiting..."),"warning")
+                 logger.warning("Lathe X and Z axis remap should be the same.
Exiting...")
                  return
            if    x.lower()+z.lower() in ["xy","yx"] : gcode_plane_selection =
"G17 (Using XY plane)\n"
            if    x.lower()+z.lower() in ["xz","zx"] : gcode_plane_selection =
"G18 (Using XZ plane)\n"
@@ -6595,7 +6559,7 @@
      def lathe_modify_path(self):
            if self.selected_paths == {} and self.options.auto_select_paths:
                  paths=self.paths
-                 self.error(_("No paths are selected! Trying to work on all
available paths."),"warning")
+                 logger.warning("No paths are selected! Trying to work on all
available paths.")
            else :
                  paths = self.selected_paths

@@ -6676,13 +6640,14 @@
                        if r :
                              ver = r.group(1).strip()
                              if ver != gcodetools_current_version :
-                                   self.error("There is a newer version of
Gcodetools you can get it at: \nhttp://www.cnc-club.ru/gcodetools (English
version). \nhttp://www.cnc-club.ru/gcodetools_ru (Russian version).
","Warning")
```

```
+                                 message = "There is a newer version of
Gcodetools you can get it at: \nhttp://www.cnc-club.ru/gcodetools (English
version). \nhttp://www.cnc-club.ru/gcodetools_ru (Russian version). "
                            else :
-                                 self.error("You are currently using latest
stable version of Gcodetools.","Warning")
+                                 message = "You are currently using latest
stable version of Gcodetools."
+                           logger.info(message)
                            return
-                self.error("Can not check the latest version. You can check it
manualy at \nhttp://www.cnc-club.ru/gcodetools (English version).
\nhttp://www.cnc-club.ru/gcodetools_ru (Russian version). \nCurrent version is
Gcodetools %s"%gcodetools_current_version,"Warning")
+                logger.error("Can not check the latest version. You can check
it manualy at \nhttp://www.cnc-club.ru/gcodetools (English version).
\nhttp://www.cnc-club.ru/gcodetools_ru (Russian version). \nCurrent version is
Gcodetools %s"%gcodetools_current_version)
           except :
-                self.error("Can not check the latest version. You can check it
manualy at \nhttp://www.cnc-club.ru/gcodetools (English version).
\nhttp://www.cnc-club.ru/gcodetools_ru (Russian version). \nCurrent version is
Gcodetools %s"%gcodetools_current_version,"Warning")
+                logger.warning("Can not check the latest version. You can
check it manualy at \nhttp://www.cnc-club.ru/gcodetools (English version).
\nhttp://www.cnc-club.ru/gcodetools_ru (Russian version). \nCurrent version is
Gcodetools %s"%gcodetools_current_version)



@@ -6790,7 +6755,7 @@
           if not self.check_dir() : return
           if self.selected_paths == {} and self.options.auto_select_paths:
                 paths=self.paths
-                self.error(_("No paths are selected! Trying to work on all
available paths."),"warning")
+                logger.warning("No paths are selected! Trying to work on all
available paths.")
           else :
                 paths = self.selected_paths
           self.tool = []
@@ -6814,7 +6779,7 @@
                                       self.graffiti_reference_points[layer] =
self.graffiti_reference_points[self.layers[i]]
                                       break
                            if reference_points == None :
-                                 self.error('There are no graffiti reference
points for layer %s'%layer,"error")
+                                 logger.error('There are no graffiti reference
points for layer %s'%layer)

                       # Transform reference points
```

```
                        for i in
range(len(self.graffiti_reference_points[layer])):
@@ -6901,8 +6866,8 @@
                                        polyline = []
                                  # max_segment_length
                                  polyline += [ sp1 ]
-                                 print_(polyline)
-                                 print_(sp1)
+                                 logger.info(str(polyline))
+                                 logger.info(str(sp1))

                                  spl = sp1
                            polyline += [ sp2 ]
@@ -6919,10 +6884,10 @@
                        for i in range(len(polylines)) :
                              polyline = []
                              l = 0
-                             print_("polylines",polylines)
-                             print_(polylines[i])
+                             logger.info("polylines %s",polylines)
+                             logger.info(str(polylines[i]))
                              for sp1,sp2 in
zip(polylines[i][1],polylines[i][1][1:]) :
-                                 print_(sp1,sp2)
+                                 logger.info("%s %s", sp1,sp2)
                                  l = cspseglength(sp1,sp2)
                                  if l>0.00000001 :
                                        polyline += [sp1[1]]
@@ -6931,7 +6896,7 @@
                                              polyline +=
[csp_at_length(sp1,sp2,float(j)/parts) ]
                              if l>0.00000001 :
                                  polyline += [sp2[1]]
-                             print_(i)
+                             logger.info(str(i))
                              polylines[i][1] = polyline

                        t = 0
@@ -6984,7 +6949,7 @@
                        f.close()

                  except :
-                       self.error("Png module have not been found!","warning")
+                       logger.warning("Png module have not been found!")



@@ -7002,39 +6967,33 @@
            options.self = self
            options.doc_root = self.document.getroot()

-           # define print_ function
```

```
-           global print_
            if self.options.log_create_log :
-                 try :
-                       if os.path.isfile(self.options.log_filename) :
os.remove(self.options.log_filename)
-                       f = open(self.options.log_filename,"a")
-                       f.write("Gcodetools log file.\nStarted at %s.\n%s\n" %
(time.strftime("%d.%m.%Y %H:%M:%S"),options.log_filename))
-                       f.write("%s tab is active.\n" % self.options.active_tab)
-                       f.close()
-                 except :
-                       print_  = lambda *x : None
-           else : print_  = lambda *x : None
+                 file_handler = logging.FileHandler(self.options.log_filename)
+                 formatter = logging.Formatter(LOG_FORMAT)
+                 file_handler.setFormatter(formatter)
+                 logger.addHandler(file_handler)
            if self.options.active_tab == '"help"' :
                  self.help()
                  return
            elif self.options.active_tab == '"about"' :
+                 self.help()
                  return

            elif self.options.active_tab ==  '"test"' :
                  self.test()

            elif self.options.active_tab not in ['"dxfpoints"','"path-to-
gcode"', '"area_fill"', '"area"', '"area_artefacts"', '"engraving"',
'"orientation"', '"tools_library"', '"lathe"', '"offset"', '"arrangement"',
'"update"', '"graffiti"', '"lathe_modify_path"', '"plasma-prepare-path"']:
-                 self.error(_("Select one of the action tabs - Path to Gcode,
Area, Engraving, DXF points, Orientation, Offset, Lathe or Tools library.\n
Current active tab id is %s" % self.options.active_tab),"error")
+                 logger.error("Select one of the action tabs - Path to Gcode,
Area, Engraving, DXF points, Orientation, Offset, Lathe or Tools library.\n
Current active tab id is %s", self.options.active_tab)
            else:
                  # Get all Gcodetools data from the scene.
                  self.get_info()
                  if self.options.active_tab in ['"dxfpoints"','"path-to-
gcode"', '"area_fill"', '"area"', '"area_artefacts"', '"engraving"', '"lathe"',
'"graffiti"', '"plasma-prepare-path"']:
                        if self.orientation_points == {} :
-                             self.error(_("Orientation points have not been
defined! A default set of orientation points has been automatically
added."),"warning")
+                             logger.warning("Orientation points have not been
defined! A default set of orientation points has been automatically added.")
                              self.orientation(
self.layers[min(1,len(self.layers)-1)] )
                              self.get_info()
```

```
                        if self.tools == {} :
-                             self.error(_("Cutting tool has not been defined! A
default tool has been automatically added."),"warning")
+                             logger.warning("Cutting tool has not been defined!
A default tool has been automatically added.")
                              self.options.tools_library_type = "default"
                              self.tools_library(
self.layers[min(1,len(self.layers)-1)] )
                              self.get_info()
@@ -7071,7 +7030,7 @@
                                    if len(self.selected_paths[layer]) == 2 :
                                          csp1, csp2 =
cubicsuperpath.parsePath(self.selected_paths[layer][0].get("d")),
cubicsuperpath.parsePath(self.selected_paths[layer][1].get("d"))
                                          dist = csp_to_csp_distance(csp1,csp2)
-                                         print_(dist)
+                                         logger.info(str(dist))
                                          draw_pointer(
list(csp_at_t(csp1[dist[1]][dist[2]-1],csp1[dist[1]][dist[2]],dist[3]))

      +list(csp_at_t(csp2[dist[4]][dist[5]-
1],csp2[dist[4]][dist[5]],dist[6])),"red","line", comment = math.sqrt(dist[0]))
                              return
@@ -7089,18 +7048,14 @@
                                          if offset_ != [] :
                                                for iii in offset_ :
                                                      draw_csp([iii],
color="Green", width=1)
-                                                     #print_(offset_)
+                                                     #logger.info(str(offset_))
                                          else :
-                                               print_("------------Reached empty
offset at radius %s"% offset )
+                                               logger.info("------------Reached
empty offset at radius %s"% offset )
                                                break
                                          offset += self.options.offset_step
-                       print_()
-                       print_("-------------------------------------------------
-----------------------------------")
-                       print_("-------------------------------------------------
-----------------------------------")
-                       print_("-------------------------------------------------
-----------------------------------")
-                       print_()
-                       print_("Done in %s"%(time.time()-time_))

-                       print_("Total offsets count %s"%offsets_count)

+                       logger.info("-------------------------------------------
-----")
```

```
+                       logger.info("Done in %s"%(time.time()-time_start))

+                       logger.info("Total offsets count %s"%offsets_count)

                  elif self.options.active_tab == '"arrangement"':
                        self.arrangement()

@@ -7108,12 +7063,10 @@
                        self.plasma_prepare_path()


-          print_("-----------------------------------------")
-          print_("Done in %f seconds"%(time.time()-start_time))
-          print_("End at %s."%time.strftime("%d.%m.%Y %H:%M:%S"))
-
-
-#
+          logger.info("Done in %f seconds"%(time.time()-start_time))
+          logger.info("End at %s."%time.strftime("%d.%m.%Y %H:%M:%S"))
+
+logger.info("-------------------------------------------------")
 gcodetools = Gcodetools()
 gcodetools.affect()


=== added file 'Makefile'
--- Makefile     1970-01-01 00:00:00 +0000
+++ Makefile     2011-12-06 09:06:25 +0000
@@ -0,0 +1,146 @@
+INPUT_SVG := test.svg
+
+FUNCTIONS := dxfpoints path-to-gcode area_fill area \
+area_artefacts engraving orientation tools_library lathe \
+offset arrangement graffiti lathe_modify_path plasma-prepare-path
+
+.PHONY: clean inx doc install uninstall patch-log-debug\
+patch-log test-all test
+
+clean:
+     @rm -rf test
+     @rm -f gcodetools_about-dev.inx
+     @rm -f gcodetools_area-dev.inx
+     @rm -f gcodetools_check_for_updates-dev.inx
+     @rm -f gcodetools_dxf_points-dev.inx
+     @rm -f gcodetools_engraving-dev.inx
+     @rm -f gcodetools_graffiti-dev.inx
+     @rm -f gcodetools_lathe-dev.inx
+     @rm -f gcodetools_orientation_points-dev.inx
+     @rm -f gcodetools_path_to_gcode-dev.inx
+     @rm -f gcodetools_prepare_path_for_plasma-dev.inx
+     @rm -f gcodetools_tools_library-dev.inx
+     make -C doc clean
```

```
+
+inx: gcodetools-dev.inx create_inx.py
+     python create_inx.py
+
+doc: doc/Makefile
+     make -C doc
+
+install: gcodetools-dev.py dxf_input.py dxf_input.inx png.py inx
+     cp gcodetools-dev.py dxf_input.py png.py ~/.config/inkscape/extensions/
+     cp dxf_input.inx ~/.config/inkscape/extensions
+     cp gcodetools_*.inx ~/.config/inkscape/extensions
+
+uninstall:
+     @rm -f ~/.config/inkscape/extensions/gcodetools_*.inx
+     @rm -f ~/.config/inkscape/extensions/gcodetools-dev.py
+     @rm -f ~/.config/inkscape/extensions/png.py
+     @rm -f ~/.config/inkscape/extensions/dxf_input.py
+     @rm -f ~/.config/inkscape/extensions/dxf_input.inx
+
+patch-log-debug: gcodetools-dev.py
+     @sed -i "s/^LOG_LEVEL = logging\..*$$/LOG_LEVEL = logging.DEBUG/" $<
+
+patch-log: gcodetools-dev.py
+     @sed -i "s@logger = logging.getLogger('gcodetools')@\
+logger = logging.getLogger('gcodetools')\n\
+debug_handler = logging.FileHandler(\"/var/log/gcodetools.debug\")\n\
+debug_handler.setFormatter(logging.Formatter(LOG_FORMAT))\n\
+logger.addHandler(debug_handler)\n@" $<
+
+test-all:
+     @for i in $(FUNCTIONS); do make test TEST_TYPE=$$i; done
+
+TEMP_REPO := /tmp/gcodetools.test
+test-rev:
+     if [ -d $(TEMP_REPO) ]; then rm -rf $(TEMP_REPO); fi
+     bzr branch ./ $(TEMP_REPO)
+     cd $(TEMP_REPO); \
+     bzr update -r $(TEST_REVISION); bzr revert -r $(TEST_REVISION);
+     if [ ! -f $(TEMP_REPO)/Makefile ]; then cp ./Makefile $(TEMP_REPO); fi
+     cp $(INPUT_SVG) $(TEMP_REPO)
+     make -C $(TEMP_REPO) test-all INPUT_SVG=`basename $(INPUT_SVG)`
+     if [ ! -d test ]; then mkdir test; fi
+     cp -r $(TEMP_REPO)/test/$(TEST_REVISION) test/
+     rm -rf $(TEMP_REPO)
+
+REVISION := $(shell bzr revno --tree)
+OUTPUT_DIR := test/$(REVISION)/`basename $(INPUT_SVG) .svg`.d
+test: gcodetools-dev.py $(INPUT_SVG) patch-log-debug
+     @echo "Running $(TEST_TYPE) on $(INPUT_SVG) for revision $(REVISION)"
+     @if [ ! -d $(OUTPUT_DIR) ]; then mkdir -p $(OUTPUT_DIR); fi
+     @export PYTHONPATH=$PYTHONPATH:/usr/share/inkscape/extensions; \
+     python gcodetools-dev.py \
```

```
+           --active-tab='"$(TEST_TYPE)"' \
+           --Zscale=1 \
+           --Zoffset=0 \
+           --auto_select_paths=true \
+           --min-arc-radius=0.05000000074505806 \
+           --comment-gcode-from-properties=false \
+           --filename=$(TEST_TYPE).ngc \
+           --add-numeric-suffix-to-filename=false \
+           --directory=$(OUTPUT_DIR) \
+           --Zsafe=5 \
+           --unit=G21\ \(All\ units\ in\ mm\) \
+           --postprocessor=''  \
+           --create-log=true \
+           --log-filename=$(OUTPUT_DIR)/$(TEST_TYPE).log \
+           --dxfpoints-action=save \
+           --biarc-tolerance=0.0099999997764825821 \
+           --biarc-max-split-depth=4 \
+           --path-to-gcode-order=subpath\ by\ subpath \
+           --path-to-gcode-depth-function=d \
+           --path-to-gcode-sort-paths=false \
+           --area-fill-angle=0 \
+           --area-fill-shift=0 \
+           --area-fill-method=zig-zag \
+           --max-area-curves=100 \
+           --area-inkscape-radius=-10 \
+           --area-tool-overlap=0 \
+           --area-find-artefacts-diameter=5 \
+           --area-find-artefacts-action=mark\ with\ an\ arrow \
+           --engraving-sharp-angle-tollerance=175 \
+           --engraving-max-dist=10 \
+           --engraving-newton-iterations=4 \
+           --engraving-draw-calculation-paths=false \
+           --orientation-points-count=2 \
+           --Zsurface=0 \
+           --Zdepth=-1 \
+           --tools-library-type=default\ tool \
+           --lathe-width=10 \
+           --lathe-fine-cut-width=1 \
+           --lathe-fine-cut-count=1 \
+           --lathe-create-fine-cut-using=Move\ path \
+           --lathe-x-axis-remap=X \
+           --lathe-z-axis-remap=Z \
+           --lathe-rectangular-cutter-width=4 \
+           --offset-radius=10 \
+           --offset-step=10 \
+           --offset-draw-clippend-path=false \
+           --offset-just-get-distance=false \
+           --arrangement-material-width=500 \
+           --arrangement-population-count=100 \
+           --arrangement-inline-test=false \
+           --in-out-path=false \
+           --in-out-path-len=10 \
```

```
+          --in-out-path-point-max-dist=10 \
+          --in-out-path-type=Round \
+          --in-out-path-radius=10 \
+          --in-out-path-replace-original-path=false \
+          --in-out-path-do-not-add-reference-point=false \
+          --plasma-prepare-corners=false \
+          --plasma-prepare-corners-distance=10 \
+          --plasma-prepare-corners-tolerance=170 \
+          --graffiti-max-seg-length=10 \
+          --graffiti-min-radius=10 \
+          --graffiti-start-pos=\(0.0\;0.0\) \
+          --graffiti-create-preview=true \
+          --graffiti-create-linearization-preview=true \
+          --graffiti-preview-size=800 \
+          --graffiti-preview-emmit=1000 \
+          $(INPUT_SVG) \
+               1> $(OUTPUT_DIR)/$(TEST_TYPE).svg \
+               2> $(OUTPUT_DIR)/$(TEST_TYPE).error;\
+     echo $$? > $(OUTPUT_DIR)/$(TEST_TYPE).status

=== modified file 'doc/Makefile'
--- doc/Makefile 2011-08-04 20:17:56 +0000
+++ doc/Makefile 2011-12-06 09:06:25 +0000
@@ -6,6 +6,9 @@

 %.html: %.asciidoc
      a2x -f xhtml -v --dblatex-opts="$(DBLATEX_OPTS)" $<
-
+
 %.odt: %.asciidoc
      a2x -f odt -v --dblatex-opts="$(DBLATEX_OPTS)" $<
+
+clean:
+     rm -f gcodetools-ru.pdf gcodetools-ru.html gcodetools-ru.odt
\ No newline at end of file

=== modified file 'gcodetools-dev.inx'
--- gcodetools-dev.inx 2011-12-06 05:27:14 +0000
+++ gcodetools-dev.inx 2011-12-06 06:16:25 +0000
@@ -110,7 +110,6 @@
           <page name='area_fill' _gui-text='Fill area'>
                <param name="area-fill-angle" type="float" min="-360"
max="360" _gui-text="Area fill angle">0</param>
                <param name="area-fill-shift" type="float" min="-1" max="1"
_gui-text="Area fill shift">0</param>
-               <param name="area-fill-method" type="float" min="-1" max="1"
_gui-text="Area fill shift">0</param>
                <param name="area-fill-method"  _gui-text="Filling method"
type="optiongroup">
                      <_option value="zig-zag">Zig zag</_option>
                      <_option value="spiral">Spiral</_option>
@@ -186,11 +185,6 @@
```

```
                </param>
                <param name="Zsurface" type="float" precision="5" min="-1000"
max="1000" _gui-text="Z surface:">0</param>
                <param name="Zdepth" type="float" precision="5" min="-1000"
max="1000" _gui-text="Z depth:">-1</param>
-               <param name="unit" type="enum" _gui-text="Units (mm or in):">
-                     <_item value="G21 (All units in mm)">mm</_item>
-                     <_item value="G20 (All units in inches)">in</_item>
-               </param>
-
                <_param name="help" type="description">
 Orientation points are used to calculate transformation
(offset,scale,mirror,rotation in XY plane) of the path.
 3-points mode only: do not put all three into one line (use 2-points mode
instead).

=== modified file 'gcodetools-dev.py'
--- gcodetools-dev.py  2011-12-06 05:27:14 +0000
+++ gcodetools-dev.py  2011-12-06 06:16:25 +0000
@@ -3810,19 +3810,19 @@

      def __init__(self):
           inkex.Effect.__init__(self)
-          self.OptionParser.add_option("-d", "--directory",
      action="store", type="string",         dest="directory", default="/home",
                              help="Directory for gcode file")
-          self.OptionParser.add_option("-f", "--filename",
      action="store", type="string",         dest="file", default="-1.0",
                       help="File name")
+          self.OptionParser.add_option("-d", "--directory",
      action="store", type="string",         dest="directory", default="/tmp",
                       help="Directory for gcode file")
+          self.OptionParser.add_option("-f", "--filename",
      action="store", type="string",         dest="file", default="output.ngc",
                                  help="File name")
           self.OptionParser.add_option("",   "--add-numeric-suffix-to-
filename", action="store", type="inkbool",
      dest="add_numeric_suffix_to_filename", default=True,help="Add numeric
suffix to filename")
           self.OptionParser.add_option("", "--load-in-emc", action="store",
type="inkbool", dest="load_into_emc", default=False, help="Load output into
AXIS on completion.")
           self.OptionParser.add_option("",   "--Zscale",
           action="store", type="float",          dest="Zscale",
default="1.0",                                help="Scale factor Z")

           self.OptionParser.add_option("",   "--Zoffset",
           action="store", type="float",          dest="Zoffset",
default="0.0",                                help="Offset along Z")
-          self.OptionParser.add_option("-s", "--Zsafe",
      action="store", type="float",          dest="Zsafe", default="0.5",
                       help="Z above all obstacles")
```

```
+           self.OptionParser.add_option("-s", "--Zsafe",
      action="store", type="float",           dest="Zsafe", default="5.0",
                        help="Z above all obstacles")
            self.OptionParser.add_option("-z", "--Zsurface",
      action="store", type="float",           dest="Zsurface", default="0.0",
                              help="Z of the surface")
            self.OptionParser.add_option("-c", "--Zdepth",
            action="store", type="float",           dest="Zdepth", default="-
0.125",                           help="Z depth of cut")
            self.OptionParser.add_option("",   "--Zstep",
      action="store", type="float",           dest="Zstep", default="-0.125",
                              help="Z step of cutting")
            self.OptionParser.add_option("-p", "--feed",
      action="store", type="float",           dest="feed", default="4.0",
                              help="Feed rate in unit/min")

-           self.OptionParser.add_option("",   "--biarc-tolerance",
      action="store", type="float",           dest="biarc_tolerance",
default="1",                      help="Tolerance used when calculating biarc
interpolation.")
+           self.OptionParser.add_option("",   "--biarc-tolerance",
      action="store", type="float",           dest="biarc_tolerance",
default="0.001",                  help="Tolerance used when calculating biarc
interpolation.")
            self.OptionParser.add_option("",   "--biarc-max-split-depth",
      action="store", type="int",       dest="biarc_max_split_depth",
default="4",                  help="Defines maximum depth of splitting while
approximating using biarcs.")
            self.OptionParser.add_option("",   "--path-to-gcode-order",
      action="store", type="string",          dest="path_to_gcode_order",
default="path by path",     help="Defines cutting order path by path or layer
by layer.")
            self.OptionParser.add_option("",   "--path-to-gcode-depth-
function",action="store", type="string",
      dest="path_to_gcode_depth_function", default="zd", help="Path to gcode
depth function.")
@@ -3834,16 +3834,16 @@

            self.OptionParser.add_option("",   "--tool-diameter",
      action="store", type="float",           dest="tool_diameter", default="3",
                              help="Tool diameter used for area cutting")
            self.OptionParser.add_option("",   "--max-area-curves",
      action="store", type="int",       dest="max_area_curves", default="100",
                  help="Maximum area curves for each area")
-           self.OptionParser.add_option("",   "--area-inkscape-radius",
      action="store", type="float",           dest="area_inkscape_radius",
default="0",                  help="Area curves overlaping (depends on tool
diameter [0,0.9])")
-           self.OptionParser.add_option("",   "--area-tool-overlap",
      action="store", type="float",           dest="area_tool_overlap",
default="-10",                help="Radius for preparing curves using inkscape")
```

```
+          self.OptionParser.add_option("",   "--area-inkscape-radius",
     action="store", type="float",          dest="area_inkscape_radius",
default="-10",              help="Area curves overlaping (depends on tool
diameter [0,0.9])")
+          self.OptionParser.add_option("",   "--area-tool-overlap",
     action="store", type="float",          dest="area_tool_overlap",
default="0",                help="Radius for preparing curves using inkscape")
           self.OptionParser.add_option("",   "--unit",
     action="store", type="string",         dest="unit", default="G21 (All
units in mm)",         help="Units")
-          self.OptionParser.add_option("",   "--active-tab",
     action="store", type="string",         dest="active_tab", default="",
                            help="Defines which tab is active")
+          self.OptionParser.add_option("",   "--active-tab",
     action="store", type="string",         dest="active_tab", default='"path-
to-gcode"',                             help="Defines which tab is active")

           self.OptionParser.add_option("",   "--area-fill-angle",
     action="store", type="float",          dest="area_fill_angle",
default="0",                            help="Fill area with lines heading this
angle")
           self.OptionParser.add_option("",   "--area-fill-shift",
     action="store", type="float",          dest="area_fill_shift",
default="0",                            help="Shift the lines by tool d *
shift")
           self.OptionParser.add_option("",   "--area-fill-method",
     action="store", type="string",         dest="area_fill_method",
default="zig-zag",                          help="Filling method either zig-
zag or spiral")

-          self.OptionParser.add_option("",   "--area-find-artefacts-
diameter",action="store", type="float",
     dest="area_find_artefacts_diameter", default="1",
     help="Artefacts seeking radius")
+          self.OptionParser.add_option("",   "--area-find-artefacts-
diameter",action="store", type="float",
     dest="area_find_artefacts_diameter", default="5",
     help="Artefacts seeking radius")
           self.OptionParser.add_option("",   "--area-find-artefacts-action",
     action="store", type="string",         dest="area_find_artefacts_action",
default="mark with an arrow",     help="Artefacts action type")

           self.OptionParser.add_option("",   "--auto_select_paths",
     action="store", type="inkbool",        dest="auto_select_paths",
default=True,                     help="Select all paths if nothing is
selected.")
@@ -3852,17 +3852,17 @@
           self.OptionParser.add_option("",   "--loft-direction",
     action="store", type="string",         dest="loft_direction",
default="crosswise",              help="Direction of loft's interpolation.")
           self.OptionParser.add_option("",   "--loft-interpolation-degree",
     action="store", type="float",          dest="loft_interpolation_degree",
```

```
default="2",          help="Which interpolation use to loft the paths smooth
interpolation or staright.")

-          self.OptionParser.add_option("",   "--min-arc-radius",
     action="store", type="float",          dest="min_arc_radius",
default=".1",                     help="All arc having radius less than minimum
will be considered as straight line")
+          self.OptionParser.add_option("",   "--min-arc-radius",
     action="store", type="float",          dest="min_arc_radius",
default="0.05",                   help="All arc having radius less than minimum
will be considered as straight line")

-          self.OptionParser.add_option("",   "--engraving-sharp-angle-
tollerance",action="store", type="float",
     dest="engraving_sharp_angle_tollerance", default="150",       help="All
angles thar are less than engraving-sharp-angle-tollerance will be thought
sharp")
+          self.OptionParser.add_option("",   "--engraving-sharp-angle-
tollerance",action="store", type="float",
     dest="engraving_sharp_angle_tollerance", default="175",       help="All
angles that are less than engraving-sharp-angle-tollerance will be thought
sharp")
           self.OptionParser.add_option("",   "--engraving-max-dist",
     action="store", type="float",          dest="engraving_max_dist",
default="10",                           help="Distanse from original path where
engraving is not needed (usualy it's cutting tool diameter)")
           self.OptionParser.add_option("",   "--engraving-newton-iterations",
action="store", type="int",       dest="engraving_newton_iterations",
default="4",                help="Number of sample points used to calculate
distance")
           self.OptionParser.add_option("",   "--engraving-draw-calculation-
paths",action="store", type="inkbool",
     dest="engraving_draw_calculation_paths", default=False,       help="Draw
additional graphics to debug engraving path")
           self.OptionParser.add_option("",   "--engraving-cutter-shape-
function",action="store", type="string",
     dest="engraving_cutter_shape_function", default="w",
     help="Cutter shape function z(w). Ex. cone: w. ")

           self.OptionParser.add_option("",   "--lathe-width",
           action="store", type="float",          dest="lathe_width",
default=10.,                                      help="Lathe width")
-          self.OptionParser.add_option("",   "--lathe-fine-cut-width",
     action="store", type="float",          dest="lathe_fine_cut_width",
default=1.,                       help="Fine cut width")
-          self.OptionParser.add_option("",   "--lathe-fine-cut-count",
     action="store", type="int",       dest="lathe_fine_cut_count",
default=1.,                       help="Fine cut count")
+          self.OptionParser.add_option("",   "--lathe-fine-cut-width",
     action="store", type="float",          dest="lathe_fine_cut_width",
default=1.0,                            help="Fine cut width")
```

```
+          self.OptionParser.add_option("",   "--lathe-fine-cut-count",
     action="store", type="int",       dest="lathe_fine_cut_count",
default=1.0,                           help="Fine cut count")
           self.OptionParser.add_option("",   "--lathe-create-fine-cut-using",
     action="store", type="string",
     dest="lathe_create_fine_cut_using", default="Move path",
     help="Create fine cut using")
           self.OptionParser.add_option("",   "--lathe-x-axis-remap",
     action="store", type="string",         dest="lathe_x_axis_remap",
default="X",                                help="Lathe X axis remap")
           self.OptionParser.add_option("",   "--lathe-z-axis-remap",
     action="store", type="string",         dest="lathe_z_axis_remap",
default="Z",                                help="Lathe Z axis remap")
@@ -3873,14 +3873,14 @@
           self.OptionParser.add_option("",   "--log-filename",
     action="store", type="string",         dest="log_filename",
default='/var/log/gcodetools.log',                          help="Create log
files")

           self.OptionParser.add_option("",   "--orientation-points-count",
     action="store", type="string",         dest="orientation_points_count",
default="2",                help="Orientation points count")
-          self.OptionParser.add_option("",   "--tools-library-type",
     action="store", type="string",         dest="tools_library_type",
default='cylinder cutter',  help="Create tools definition")
+          self.OptionParser.add_option("",   "--tools-library-type",
     action="store", type="string",         dest="tools_library_type",
default='default tool',     help="Create tools definition")

           self.OptionParser.add_option("",   "--dxfpoints-action",
     action="store", type="string",         dest="dxfpoints_action",
default='replace',               help="dxfpoint sign toggle")


           self.OptionParser.add_option("",   "--help-language",
     action="store", type="string",         dest="help_language",
default='http://www.cnc-club.ru/forum/viewtopic.php?f=33&t=35',    help="Open
help page in webbrowser.")

-          self.OptionParser.add_option("",   "--offset-radius",
     action="store", type="float",          dest="offset_radius", default=10.,
           help="Offset radius")
-          self.OptionParser.add_option("",   "--offset-step",
           action="store", type="float",          dest="offset_step",
default=10.,          help="Offset step")
+          self.OptionParser.add_option("",   "--offset-radius",
     action="store", type="float",          dest="offset_radius",
default=10.0,         help="Offset radius")
+          self.OptionParser.add_option("",   "--offset-step",
           action="store", type="float",          dest="offset_step",
default=10.0,         help="Offset step")
```

```
           self.OptionParser.add_option("",   "--offset-draw-clippend-path",
      action="store", type="inkbool",        dest="offset_draw_clippend_path",
default=False,         help="Draw clipped path")
           self.OptionParser.add_option("",   "--offset-just-get-distance",
      action="store", type="inkbool",        dest="offset_just_get_distance",
default=False,         help="Don't do offset just get distance")

@@ -3889,29 +3889,29 @@
           self.OptionParser.add_option("",   "--arrangement-inline-test",
      action="store", type="inkbool",   dest="arrangement_inline_test",
default=False,   help="Use C-inline test (some additional packets will be
needed)")


-          self.OptionParser.add_option("",   "--postprocessor",
      action="store", type="string",         dest="postprocessor", default='',
           help="Postprocessor command.")
+          self.OptionParser.add_option("",   "--postprocessor",
      action="store", type="string",         dest="postprocessor", default=' ',
                 help="Postprocessor command.")
           self.OptionParser.add_option("",   "--postprocessor-custom",
      action="store", type="string",         dest="postprocessor_custom",
default='',help="Postprocessor custom command.")

-          self.OptionParser.add_option("",   "--graffiti-max-seg-length",
      action="store", type="float",          dest="graffiti_max_seg_length",
default=1.,help="Graffiti maximum segment length.")
-          self.OptionParser.add_option("",   "--graffiti-min-radius",
      action="store", type="float",          dest="graffiti_min_radius",
default=10.,     help="Graffiti minimal connector's radius.")
+          self.OptionParser.add_option("",   "--graffiti-max-seg-length",
      action="store", type="float",          dest="graffiti_max_seg_length",
default=10.0,    help="Graffiti maximum segment length.")
+          self.OptionParser.add_option("",   "--graffiti-min-radius",
      action="store", type="float",          dest="graffiti_min_radius",
default=10.0,    help="Graffiti minimal connector's radius.")
           self.OptionParser.add_option("",   "--graffiti-start-pos",
      action="store", type="string",         dest="graffiti_start_pos",
default="(0;0)", help="Graffiti Start position (x;y).")
           self.OptionParser.add_option("",   "--graffiti-create-
linearization-preview",     action="store", type="inkbool",
      dest="graffiti_create_linearization_preview", default=True,
      help="Graffiti create linearization preview.")
           self.OptionParser.add_option("",   "--graffiti-create-preview",
      action="store", type="inkbool",   dest="graffiti_create_preview",
default=True,    help="Graffiti create preview.")
           self.OptionParser.add_option("",   "--graffiti-preview-size",
      action="store", type="int",       dest="graffiti_preview_size",
default=800,     help="Graffiti preview's size.")
-          self.OptionParser.add_option("",   "--graffiti-preview-emmit",
      action="store", type="int",       dest="graffiti_preview_emmit",
default=800,     help="Preview's paint emmit (pts/s).")
```

```
+           self.OptionParser.add_option("",   "--graffiti-preview-emmit",
      action="store", type="int",       dest="graffiti_preview_emmit",
default=1000,    help="Preview's paint emmit (pts/s).")


            self.OptionParser.add_option("",   "--in-out-path",
            action="store", type="inkbool",   dest="in_out_path",
      default=True,                help="Create in-out paths")
            self.OptionParser.add_option("",   "--in-out-path-do-not-add-
reference-point",     action="store", type="inkbool",
dest="in_out_path_do_not_add_reference_point", default=False, help="Just add
reference in-out point")
-           self.OptionParser.add_option("",   "--in-out-path-point-max-dist",
      action="store", type="float",          dest="in_out_path_point_max_dist",
default=10.,     help="In-out path max distance to reference point")
+           self.OptionParser.add_option("",   "--in-out-path-point-max-dist",
      action="store", type="float",          dest="in_out_path_point_max_dist",
default=10.0,    help="In-out path max distance to reference point")
            self.OptionParser.add_option("",   "--in-out-path-type",
      action="store", type="string",         dest="in_out_path_type",
default="Round", help="In-out path type")
-           self.OptionParser.add_option("",   "--in-out-path-len",
      action="store", type="float",          dest="in_out_path_len",
default=10.,           help="In-out path length")
+           self.OptionParser.add_option("",   "--in-out-path-len",
      action="store", type="float",          dest="in_out_path_len",
default=10.0,          help="In-out path length")
            self.OptionParser.add_option("",   "--in-out-path-replace-original-
path",action="store", type="inkbool", dest="in_out_path_replace_original_path",
default=False,   help="Replace original path")
-           self.OptionParser.add_option("",   "--in-out-path-radius",
      action="store", type="float",          dest="in_out_path_radius",
default=10.,           help="In-out path radius for round path")
+           self.OptionParser.add_option("",   "--in-out-path-radius",
      action="store", type="float",          dest="in_out_path_radius",
default=10.0,          help="In-out path radius for round path")

            self.OptionParser.add_option("",   "--plasma-prepare-corners",
      action="store", type="inkbool",        dest="plasma_prepare_corners",
default=True,    help="Prepare corners")
-           self.OptionParser.add_option("",   "--plasma-prepare-corners-
distance", action="store", type="float",
      dest="plasma_prepare_corners_distance", default=10.,help="Stepout
distance for corners")
-           self.OptionParser.add_option("",   "--plasma-prepare-corners-
tolerance", action="store", type="float",
      dest="plasma_prepare_corners_tolerance", default=10.,help="Maximum angle
for corner (0-180 deg)")
+           self.OptionParser.add_option("",   "--plasma-prepare-corners-
distance", action="store", type="float",
      dest="plasma_prepare_corners_distance", default=10.0,help="Stepout
distance for corners")
```

```
+           self.OptionParser.add_option("",   "--plasma-prepare-corners-
tolerance", action="store", type="float",
      dest="plasma_prepare_corners_tolerance", default=170.0,help="Maximum
angle for corner (0-180 deg)")

            self.default_tool = {
                            "name": "Default tool",
```